

Evaluating Open-source LLMs for Privacy Policy Annotation

by

Janis Hahn

Matriculation Number 456481

A thesis submitted to

Technische Universität Berlin
School IV - Electrical Engineering and Computer Science
Department of Telecommunication Systems
Service-centric Networking

Bachelor's Thesis

September 30, 2024

Supervised by:
Prof. Dr. Axel Küpper

Assistant supervisor:
Thomas Cory

Eidestattliche Erklärung / Statutory Declaration

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig und eigenhändig sowie ohne unerlaubte fremde Hilfe und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe.

I hereby declare that I have created this work completely on my own and used no other sources or tools than the ones listed.

A handwritten signature in black ink that reads "Janis Hahn". The signature is written in a cursive style with a large, looped 'J' and a distinct 'H'.

Berlin, September 30, 2024

Janis Hahn

Abstract

This thesis evaluates the performance of open-source large language models (LLMs) in annotating privacy policies with regards to GDPR compliance. With the rise of advanced closed-source LLMs like OpenAI's GPT-series of models and Anthropic's Claude models in recent years, their use in privacy policy analysis has become a popular technique for automatically processing the typically long and incomprehensible legal documents. In contrast to closed-source models, this thesis focuses on exploring the use of open-source alternatives, including Meta's Llama and Google's Gemma models. The study introduces a pipeline for the automated evaluation of these models, examining metrics such as precision, recall and F1-Score alongside semantic similarity scores and classification metrics. The evaluation of the models against a reference dataset generated by Openai's GPT-4o reveals that open-source models perform adequately for the task of the task of privacy policy annotation, but leave a lot to be wished for in terms of reliability, especially for the legal task at hand where accuracy is crucial. They also offer up great potential for a variety of applications, be it the local analysis of policies on low-end, unspecialized hardware, or the large scale analysis with mid-sized or high parameter count models on more sophisticated, dedicated GPUs. However, limitations in processing power and inherent variability in LLM outputs and privacy policies themselves present challenges. Ultimately, this thesis concludes that Google's Gemma 2 9B performs the best for privacy policy annotation among the tested models, with Gemma 2 27B showing similar performance. They currently have weaknesses, but given more developments, such as the use of other, more suitable model architectures or specialization through finetuning of models, the use of open-source LLMs has significant potential in the field of automated privacy policy annotation.

Zusammenfassung

In dieser Arbeit wird die Leistung von Open-Source-Sprachmodellen (LLMs) bei der Annotation von Datenschutzrichtlinien im Hinblick auf die Einhaltung der GDPR bewertet. Mit dem Aufkommen fortschrittlicher Closed-Source-LLMs wie der GPT-Modellserie von OpenAI und den Claude-Modellen von Anthropic in den letzten Jahren ist ihre Verwendung in der Analyse von Datenschutzrichtlinien zu einer beliebten Technik für die automatische Verarbeitung der typischerweise langen und unverständlichen juristischen Dokumente geworden. Im Gegensatz zu den Closed-Source-Modellen konzentriert sich diese Arbeit auf die Erforschung der Verwendung von Open-Source-Alternativen, einschließlich der Llama-Modellserie von Meta und den Gemma-Modellen von Google. Die Arbeit stellt eine Pipeline für die automatisierte Evaluierung dieser Modelle vor, wobei Metriken wie Präzision, Recall und F1-Score neben semantischen Ähnlichkeitswerten und Klassifikationsmetriken untersucht werden. Die Evaluierung der Modelle anhand eines Referenzdatensatzes, der von OpenAI's GPT-4o generiert wurde, zeigt, dass die Open-Source-Modelle für die Aufgabe der Annotation von Datenschutzrichtlinien angemessen sind, aber mit Bezug auf die Zuverlässigkeit viel zu wünschen übrig lassen, insbesondere für die vorliegende juristische Aufgabe, bei der die Genauigkeit entscheidend ist. Sie bieten auch ein großes Potenzial für eine Vielzahl von Anwendungen, sei es die lokale Analyse von Richtlinien auf schwächerer, nicht spezialisierter Hardware oder die groß angelegte Analyse mit mittelgroßen bis großen Modellen auf anspruchsvolleren, dedizierten GPUs. Allerdings stellen die begrenzte Rechenleistung und die intrinsische Variabilität der LLM-Ausgaben und der Datenschutzrichtlinien selbst eine Herausforderung dar. Letztendlich kommt diese Arbeit zu dem Schluss, dass Google's Gemma 2 9B unter den getesteten Modellen die beste Leistung für die Annotation von Datenschutzrichtlinien aufzeigt, wobei Gemma 2 27B eine ähnliche Leistung zeigt. Sie weisen derzeit Schwächen auf, aber angesichts weiterer Entwicklungen, wie der Verwendung anderer, besser geeigneter Modellarchitekturen oder der Spezialisierung durch Finetuning der Modelle, hat die Verwendung von Open-Source-LLMs erhebliches Potenzial im Bereich der automatisierten Annotation von Datenschutzrichtlinien.

Contents

1	Introduction	1
2	Related Work	2
2.1	NLP	2
2.2	Machine Learning	3
2.3	Automated Privacy Policy Analysis	5
3	Concept and Design	9
3.1	Research Gap	9
3.2	Goals	9
3.3	Objectives	10
3.4	Requirements	10
3.5	Methodology	12
3.6	Limitations of the Concept	14
4	Implementation	17
4.1	Dependencies	17
4.2	Prerequisites	19
4.3	Pipeline	21
4.3.1	Accumulation of Privacy Policies	21
4.3.2	Data Preprocessing	22
4.3.3	Annotation	24
4.3.4	Reviewing	27
4.4	Limitations	29
4.4.1	Hardware Limitations	29
4.4.2	Software Limitations	29
5	Evaluation	31
5.1	Methodology	31
5.1.1	Embedding Model Finetuning	31
5.1.2	Data Preprocessing	32
5.1.3	Annotation Comparison	32
5.1.3.1	FastText Embeddings	32
5.1.3.2	Cosine Similarity	32
5.1.3.3	Optimal Matching Algorithm	33
5.1.4	Metrics Calculation	33
5.1.4.1	True Positives (TP), False Positives (FP), and False Negatives (FN)	33
5.1.4.2	Correct Requirements Ratio	34
5.1.4.3	Precision, Recall, and F1 Score	34

5.1.5	Handling Unmatched Annotations	34
5.2	Results	35
5.2.1	Unreviewed Annotations	35
5.2.1.1	Precision, Recall and F1-Score	35
5.2.1.2	F1 Score Sorted by Model Size	37
5.2.1.3	Average Similarity Scores	38
5.2.1.4	Average Correct Annotations Ratio	42
5.2.1.5	Confusion Matrices	42
5.2.1.6	Combined Visualizations	45
5.2.2	Reviewed Annotations	48
5.2.2.1	Precision, Recall and F1-Score	48
5.2.2.2	F1-Score Sorted by Model Size	48
5.2.2.3	Average Similarity Scores	48
5.2.2.4	Average Correct Annotations Ratio	50
5.2.2.5	Confusion Matrices	50
5.2.2.6	Combined Visualizations	50
5.3	Discussion	51
5.3.1	Limitations and Potential Sources of Error	53
6	Conclusion	54
	List of Tables	56
	List of Figures	57
	Bibliography	58
	Appendices	63
	Appendix 1: Listings	64
1	Unannotated Passage	64
2	Annotated Passage	65
3	Reviewed Passage	67
	Appendix 2: Tables	69
1	Complete Metrics	69
	Appendix 3: Figures	71
1	Performance Metrics (Reviewed Annotations)	71
2	F1-Score Sorted by Model Size	72
3	Average Similarity Scores	73
4	Correct Requirements Ratio (Reviewed Annotations)	74
5	Confusion Matrices (Reviewed Annotations)	75
6	Combined Metrics	77

1 Introduction

The rapid growth in volume that digital data displays nowadays necessitates robust mechanisms for its organization and analysis. Privacy policies, which define how personal data is collected, processed, and stored, are critical in this regard. However, the complexity and verbosity of these documents often make them incomprehensible to the less savvy user, who might lack both the expertise to understand these documents as well as the time to read them at all. These issues pose significant challenges in ensuring transparency and compliance with regulations such as the General Data Protection Regulation (GDPR). This thesis aims to evaluate the performance of open-source large language models (LLMs) in annotating privacy policies, with a specific focus on identifying relevant GDPR principles.

The GDPR was implemented in the EU in 2018 and was a milestone for data privacy around the world. Many other countries were quick to follow with their own regulations that were strongly inspired by the GDPR. Brazil's Lei Geral de Proteção de Dados (LGPD), for example, which came into effect in 2020, has large similarities with the the European regulation, though with less severe financial penalties and different valid bases for processing [1]. The GDPR's influence continues to be tangible globally, with it being the basis for record fines against Meta in 2023 [2], for example. Such a strong regulatory tool is of no use, however, if it is not enforced consistently. Automated compliance checks could be extraordinarily beneficial for the general public by making it much more difficult for companies to slip the net of the GDPR. LLMs offer a new and promising approach to this problem that people have tried to solve in the past using other natural language processing (NLP) techniques [3].

Previous research on this topic before the popularization of large language models relied on more conventional NLP techniques, including statistical approaches and machine learning methods. The NLP toolkit NLTK [4] saw widespread use since it provides a multitude of features, one example being sentiment analysis with the rule-based VADER tool. General text annotation tasks were often done with a number of different neural network architectures. However, in recent years, large language models (LLMs) have emerged as powerful tools in natural language processing, capable of processing and generating human-like text. The introduction of GPT-3 in 2021 and subsequently ChatGPT in 2022 marked the beginning of a new wave of enthusiasm for transformer-based LLMs. These models hold substantial potential for automating the annotation and analysis of complex legal documents, such as privacy policies, thereby enhancing their accessibility and ensuring regulatory compliance. Although closed-source models such as GPT-4o show very promising results in preliminary tests, the use of open-source LLMs, such as models from the Meta Llama family, presents an attractive alternative due to their cost-effectiveness, transparency, and adaptability, which is what this project aims to investigate. Furthermore, almost all modern research uses the datasets OPP-115 and PPGDPR, which leaves an opening for us to investigate the performance of LLMs on a new, self-gathered dataset.

2 Related Work

2.1 NLP

Natural Language Processing (NLP) is a field of research that focuses on enabling computers to understand, manipulate and interact natural language text or even speech. NLP incorporates quite a few scientific fields, from linguistics, over mathematics to artificial intelligence and psychology [5]. This field has evolved significantly from its early days of rule-based and symbolic approaches. Early efforts focused on rule-based systems to process texts by applying predefined linguistic rules. ELIZA, for example, which was developed by Joseph Weizenbaum in 1966, is a natural language processing program that simulates conversation by analyzing user input and generates responses through predefined rules [6]. It identifies these keywords and context using pattern matching, and applies transformations triggered by keywords in the input to generate responses to the input. ELIZA served as an inspiration for other NLP tools, one of which is the programming assistance tool PiE, which is able to synthesize LOGO programs from conversations [7]. These approaches, while pioneering, were limited by their inability to adapt to the diverse and evolving nature of language. Also, semantic ambiguities and nuances in human language such as sarcasm posed a significant challenge for early models. World knowledge could also not be integrated into outputs, leading to a lack of understanding of idiomatic expressions, for example. Statistical methods, while not perfect, offered a more generally applicable approach. Techniques such as the use of n-grams to predict the likelihood of a word given its preceding n-1 words and Hidden Markov Models (HMMs) for part-of-speech tagging allowed for more flexibility and accuracy. In 1997, Moens and Uyttendaele demonstrated the use of shallow statistical methods for indexing legal documents, which significantly improved the retrieval and classification of legal texts [8].

With advancements in computational power, machine learning models, particularly supervised learning algorithms, became more and more popular. Support Vector Machines (SVMs) were one approach to this. They are particularly useful for classification tasks, specifically binary classification, and are effective in high-dimensional spaces, which makes them suitable for sentiment analysis [9] and text classification [10] [11] within the realm of NLP, but also image classification tasks such as face detection [12]. They tend to be robust to overfitting, which made them quite popular compared to algorithms that were more prone to such biases. Logistic Regression (LR) models were also widely used for text classification tasks, but also other more diverse applications, such as medical diagnosis [13]. However, both of these approaches come with their own drawbacks. For instance, logistic regression, when used for the purpose of text annotation, these models require large annotated datasets for training, which are often labor-intensive to gather. Both of these models also struggle with complex datasets. Logistic regression also runs into issues when the input features interact in a non-linear fashion, which is the case for more complex data [14]. This makes it unsuitable for the simulation of human-like

conversation, and SVMs have similar problems with complex data.

2.2 Machine Learning

The introduction of neural networks brought further advancements in the space of NLP. These models can capture more complex patterns in data compared to the traditional statistical methods. One of the earliest neural network designs was the perceptron, first implemented in 1957 by Frank Rosenblatt [15] which acts as a binary classifier. This architecture was continually augmented and improved, until the next significant breakthrough in the form of the incorporation of backpropagation in the neural nets, leading to so-called multilayer perceptrons [16]. These models are able to deal with non-linear and high-dimensional data, which is a significant improvement over single layer perceptrons.

At a similar time, Recurrent Neural Networks (RNNs) emerged, which incorporate connections within the net that loop back on themselves, creating a sort of natural feedback loop that allows these models to have a form of memory. However, since recurrent neural networks struggle with the vanishing gradient problem [17], which leads to inefficient learning of long-term dependencies, work was needed to solve this. Due to the nature of backpropagation through activation functions such as the sigmoid function [18], RNNs experience the diminishing of gradients passed through the layers of the net that represent long-term information, to the point where these gradients are negligible after a certain time and information is therefore lost. The most significant result of research that tried to mitigate these effects is the Long Short-Term Memory (LSTM) network. This is a type of RNN, pioneered by Sepp Hochreither and Jürgen Schmidhuber in 1997 [19], that explicitly regulates the flow of information during backpropagation in a way that also permits information to be passed through the net without being diminished. This was an essential step for language processing, since human texts often contain long-term dependencies such as references to things that happened earlier in a story, for example.

Another popular form of neural network architecture used in part for NLP, but more widely known for its applications in computer vision and pattern recognition is the Convolutional Neural Network (CNN). The basic idea of a neural network with convolutional layers was first introduced in 1980 by Kunihiko Fukushima in the form of the Neocognitron [20]. This is considered to be the precursor to modern CNNs. The convolutional layers apply filters to local regions of the input data and thereby are able to capture local dependencies and spatial hierarchies within the data. This leads to reliable pattern recognition in images, for instance, while being unaffected by a shift in position within the image. While this makes them suitable for image classification, as shown by Yann LeCun in 1989 in his work on LeNet-5, which was used to recognize handwritten digits [21], CNNs are not very well suitable for analyzing large corpora of text. However, they have shown excellent results when used for sentiment analysis, as well as for sentence modeling, as shown by Kalchbrenner et al. in 2014 [22], since local relationships within sentences can be captured well by this network architecture. RNNs are still better suitable for more classical time series modeling, though.

Arguably one of the most important modern techniques in NLP gained traction in the early 2000s, where researchers started experimenting with word embeddings in the form of vectors. Researchers demonstrated the applicability of vector representations for text in 2002 [23] and 2004 [24]. Here, words are represented as vectors in high-dimensional space, where words

that are semantically similar or are used in similar contexts are closer together, hereby enabling a comprehensive modeling of complex relationships. Word embeddings can then be used to either augment other machine learning algorithms and their understanding of text, or as a standalone tool for synonym detection, for instance. Vector word embeddings were much more widely popularized with the publication of 3 papers by Mikolov et al. in 2013 [25] [26] [27] which introduced the Word2Vec models. They revolutionized the way word representations are learned in modern NLP by providing a broadly applicable and efficient strategy for word embedding. The ability of the Word2Vec models to scale efficiently with large and diverse datasets enabled further research on thus far implausibly large corpora of text to produce high-quality embeddings. Word2Vec's capability to capture not only semantic relationships between word, but also syntactic relationships given a large enough dataset, combined with its efficiency is what makes it invaluable to modern NLP.

Shortly after, in 2014, another breakthrough paper was published by Ilya Sutskever and colleagues that demonstrated the use of deep LSTM models in a encoder-decoder configuration for a translation task [28]. Their paper on Sequence to Sequence (Seq2Seq) learning combined the latest advances in NLP by using vector embeddings for words and multilayer LSTM models to generate a new text (sequence) based on an input. Especially the Seq2Seq model's ability to handle long sentences was an unexpected but important innovation. They were able to outperform a model proposed by Bahdanau et al. that was based on RNNs [29] in reference to work done in the previous year by Kalchbrenner and Blunsom [30], where RNNs were used in conjunction with CNNs for a similar translation task. Bahdanau and colleagues introduced one more significant concept though: they used an attention mechanism to combat the issue of the model "losing interest" in certain portions of the input and therefore falsifying the output. In longer inputs, information given in the input and stored in the hidden state of the encoder is lost to the decoder. The attention mechanism used in the aforementioned paper assigns soft weights that can change during the use of the model to portions of the input, mimicking human attention to important parts of texts. This reduces the amount of information the decoder has to process and enabling the handling of even longer sentences while ensuring that important information is not lost and is another key development towards modern NLP.

The introduction of the transformer model in 2017 by Vaswani et al. [31] revolutionized NLP by bringing a new technology to the table that is capable of capturing a thus far unimaginable depth within natural language and enabling the development of huge transformer-based language models which leverage self-attention to handle long-range relationships of tokens in text. This removes the need for recurrence and convolution completely and greatly reduces the computational complexity, as well as improving the handling of long-range dependencies even more. The key improvement here, though, is in the model's capacity for parallelization. Previous approaches were hard to run in parallel, significantly impacting performance. Self-attention in transformers, however, works by computing attention weights for all tokens at the same time, in contrast to previous approaches where weights were computed one at a time for each token and depended on the previous token's computation, thereby limiting parallelization. This improvement allowed for much more efficient training of the models and faster computation of results.

The development of modern Large Language Models (LLMs), beginning with the introduction of BERT in 2018 [32], a hallmark LLM that interestingly uses an encoder-only architecture without a decoder, marked a new era in NLP. Newer models such as GPT-3 [33] and GPT-4

[34] have further pushed the widespread popularization and use of LLMs. These models, with their immense parameter counts and extensive training on enormous datasets, demonstrate remarkable capabilities in generating coherent and contextually relevant text that is (near) indistinguishable from human-written text. GPT-3 showed strong performance in few-shot learning scenarios, where the model could generalize from a few examples provided in the prompt [35]. Even newer models from 2024, namely the Llama 3.1 series of models by Meta [36], open up even more possibilities for developers due to their open source nature. These models can be downloaded and run or fine-tuned locally, enabling more data privacy compared to using the OpenAI models, for instance.

2.3 Automated Privacy Policy Analysis

One of the first documented works on personal data privacy dates back to late 1890, when Samuel Warren and Louis Brandeis wrote an article for the Harvard Law Review [37] in which they argued for the recognition of a legal right to privacy, which was a pioneering step on the way to the privacy regulations we know nowadays. They state that the right to privacy is deducible from existing legislation and should therefore be logically given as a simple extension of the law.

Almost a decade later, the US Secretary's Advisory Committee on Automated Personal Data Systems first proposed the Fair Information Practice Principles (FIPPs) as a reaction to the explosive developments in data collection [38]. This laid further groundwork for modern privacy laws, such as the EU Data Protection Directive, which was introduced in 1995 and later replaced with the more modern, overhauled GDPR in 2018. Now, regulations mandate that companies have privacy policies declaring how and for what purpose personal user data is gathered by their services. Naturally, there have been attempts to automatically process these complex privacy policies for a multitude of purposes.

In 2013, a group of researchers founded the Usable Privacy Policy Project [39], a project aimed at promoting comprehensibility and usefulness of privacy policies, which is extraordinarily low for regular Internet users due to their complex jargon and length. In their paper, they propose frameworks that leverage NLP and statistical analysis to automatically evaluate policies and lay the groundwork many other researchers in this field.

Most modern research into automated privacy policy analysis bases its work on the OPP-115 corpus of freely available privacy policies. This dataset was created in 2016 by Wilson et al. as a part of the Usable Privacy Policy Project [40]. They gathered the privacy policies in a more easily processable format and gathered annotations for them from human annotators, after which they attempted some analysis of the privacy policies based on these human-written annotations. For example, they experimented with assigning labels to segments of privacy policies in order to simplify the human annotation process.

In the same year, one of the contributing researchers of the Usable Privacy Policy Project paper, Florian Schaub, was part of another paper that investigated specifically the automated analysis of privacy requirements for mobile apps [41]. Here, in contrast to a lot of other adjacent research, static code analysis of apps played a role in the automated evaluation of privacy compliance. Similarly to other teams, though, they used machine learning models as classifiers to automatically evaluate policies. In 2017, Schaub again was part of a research team that looked

into using rudimentary classifier models such as n-gram models and syntactic parsing to extract specifics about user choice in privacy policies [42]. They mainly worked on highlighting whether or not users had a choice in how their data was gathered and used and extracting this information from the policies in order to help users make better informed choices on how and whether to use certain apps.

Frequently, researchers have had trouble finding a universally applicable approach or algorithm and therefore resorted to semi-automated approaches as the researchers that created the OPP-115 corpus did. As sophisticated LLMs were not yet widely available, proven approaches typically used expert models and had a clear separation of tasks: one model was used for classification, a different one for semantic segmentation, etc. Researchers Audich et al. attempted such a semi-automated approach in 2018 [43], albeit mainly for cost reasons, and used a supervised learning algorithm to extract keywords from privacy policies. This was shown to be more accurate than unsupervised learning, but also not as accurate as human annotators. Wilson et al. also published research in the same year that took a similar approach to the policy annotation task by utilizing machine more rudimentary NLP approaches to help human annotators with their task, therefore creating a semi-automated approach. They also explored using logistic regression, support vector machines and convolutional neural networks to classify segments and sentences within the policies. SVMs achieved the best scores across categories, while CNNs were shown to be good under certain circumstances, but unreliable under others. Specifically with CNNs, the researchers ran into issues with the model not being able to capture the full scope of certain sections of the policies due to their length, which upcoming LLMs would greatly improve on. Also in 2018, Liu et al. used the OPP-115 corpus to train CNNs, logistic regression models and SVMs to classify privacy policy text into different categories [44]. Here, no crowd-sourced annotations were used to augment the dataset, with more of a focus on the viability of a fully automated approach.

Florian Schaub of the Usable Privacy Policy Project also contributed to another piece of research in 2018 that had a broader scope than the previously discussed papers. The Polisip paper [45] took a more general approach to privacy policy annotation, with its focus lying not only on general segment-level annotations, but more fine-grained analysis and classification as well. It goes as far as to introduce a new privacy-centric language model trained on a large store of privacy policies, the main application of which is their PriBot system which can answer free-form questions about policies. After extensive preprocessing, this research also made use of CNNs for classification, as well as vector word embeddings for their inputs. Another approach taken by other researchers aimed to build a browser extension that can provide graphical highlighting of the risk level of certain portions of any opened privacy policy. The PrivacyCheck extension [46] uses the Google Prediction API to train classification models that determine the risk level for individual segments of the given policy depending on how much personal data is gathered and how it is used. For a more accurate output, the researchers defined 10 questions that relate to data privacy practices and used machine learning models to decide whether the given segment discusses a certain topic or not. This is then used in a summary that is shown to the user via the extension and should help users assess the risks of using a site and the service provider's trustworthiness.

In 2019, a team of researchers published two papers in tandem [47] [48] that present the MAPS framework and discuss how NLP can be used in order to specifically perform a large scale analysis of privacy policy compliance by scaling the evaluation to over one million apps.

They find that many Android apps have discrepancies between their declared privacy policies and the way they actually gather and process user data. Their MAPS framework combines machine learning in the form of different classifiers and static code analysis to efficiently find potential evidence of non-compliance with privacy regulations within apps. Their work, in a little bit of a contrast to a majority of related work, is based on their own custom APP-350 corpus of privacy policies, which they also made available for further use as a part of the Usable Privacy Policy Project. A paper published in 2020 [49] focused on mobile health apps as a subset of the more general Android apps discussed in the 2019 MAPS papers in a bid to direct more attention towards apps that gather crucial personal data. They uncovered significant gaps in the privacy policies declared by the apps they examined, ranging from insecure data transmissions up to incomplete or even fully missing privacy policies. They developed the HPDroid tool for the automated analysis of privacy compliance for apps that, similarly to the previously outlined MAPS framework, uses a mix of ML classifiers and static code analysis to expose these privacy risks.

Researchers Yu et al. proposed their PPChecker framework in 2021 [50] which also takes the approach of combining machine learning techniques and static app code analysis in order to identify five specific potential issues with privacy policies. They define these five categories as completeness, correctness, precision, consistency and user-friendly. Narrowing down potential areas of non-compliance helps classifier models make more precise decisions. The researchers also worked on more fine-grained analysis compared to previous efforts, delving into sentence-level analysis and even going as far as to analyze third-party libraries that were used in the apps to ensure upstream compliance with privacy requirements as well. Another paper published in the same year by Bui et al. employs even more modern technologies in the form of Named Entity Recognition (NER) and using BERT to create contextualized word embeddings for their neural networks. Using an augmented version of the OPP-115 corpus as a training dataset, they created the PI-Extract System, which achieves significantly better performance than previous approaches in extracting fine-grained data practices from policies. This is just one of quite a few papers written around this time that signify a shift in interest towards recently popularized transformer-based language models due to their substantially improved ability to handle long-range context, thus allowing for much broader and more precise processing of policy texts. Towards the end of 2021, work was published on a tool named PurPliance [51] that focused on finding inconsistencies in the declared use cases of personal data within the policies themselves, as well as using the somewhat novel approach of catching and analyzing network traffic to detect further discrepancies between the declared usages of the collected data and where it is truly sent. This research also utilized neural NLP models for Semantic Role Labeling (SRL), one of which was also BERT-based, to accurately capture contradictions in the investigated policies. During the same conference the PurPliance paper was publicized at, a separate team of researchers presented their TLDR pipeline [52] which leaned even more on machine learning as a tool for natural language analysis in the form of CNNs, BERT and different document/word vector encoders. Two researchers from this team continued their work in a 2022 paper they published subsequently [53] in which they used the TLDR pipeline they had introduced in their prior work and used it to compare the data practices of paid vs. free content websites. TLDR pushed the use of LLMs as general annotators as opposed to most prior work, which mostly centered on using machine learning models as classifiers, thereby leveraging much more of modern LLMs' potential.

Research on the machine-readable representation of privacy policies was also conducted at TU Berlin by Grünewald and Pallas [54], where they introduced a formal language as well as accompanying toolkit named TILT for the purpose of managing and presenting GDPR-required transparency information in a more structured way instead of the more common natural language documents provided by most services. This work is further built upon by them in a 2023 paper [55] in which they present a human-in-the-loop approach for privacy policy annotation, which is closely aligned with our goals for this project. They extract information from privacy policies and generate sentence-level annotations using specialized BERT models, utilizing their TILT pipeline to convert the annotations into their machine-friendly format. They compare a fine-tuned version of BERT for binary classification tasks, SentenceBERT (SBERT), which is an addition to BERT optimized for handling sentences, and a naive Bayes classifier combined with static word embeddings for the annotation task, coming to the conclusion that SBERT yields the best results.

Some of the most modern research by Tang et al. introduced PolicyGPT [56], a framework for using LLMs to analyze and annotate privacy policies. Similar to our own goal, their work demonstrates the proficiency of large language models when given the task of analyzing privacy policies. They compare the performance of ChatGPT, GPT-4 and Claude 2 for text classification in the context of policy annotation. Other recent research into the possible use of LLMs for this purpose demonstrate promising results when using auxiliary technologies such as retrieval augmented generation (RAG) to help mitigate typical LLM "hallucinations" [57]. Background information that is designed to help the LLM make more well-informed and precise decisions is saved in a vector store and used as additional input. RAG is a performant, time-saving and flexible alternative to fine-tuning, as it produces comparable results while avoiding costly retraining of models when regulations are updated, for example. However, in their work, the LLM is only used to extract information from legal texts, which is then used to populate a knowledge graph. The knowledge graph is used for the main part of reasoning. The 2024 paper "Measuring privacy policy compliance in the Alexa ecosystem: In-depth analysis" also makes use of a language model. However, in contrast to the previously outlined papers, this project uses a fine-tuned version of BERT for NER instead of other LLMs that offer a more broad use case, but are potentially more prone to inaccuracies.

3 Concept and Design

3.1 Research Gap

As evidenced by prior research, privacy policies have presented a challenge for the average end user of online services for a while now, and while privacy regulations like the GDPR aim to help protect peoples’ rights online, they do have the side effect of making the rights people do and don’t have in regard to data privacy more difficult to comprehend. LLMs have gained popularity for the purpose of automated evaluation of these policies, but most of the existing research focuses on the use of closed-source models for their reliability and the quality of their output. This thesis aims to investigate the viability of using open-source LLMs at all, and to empirically find the best candidate among the most popular open models. These have the main benefit of being more cost effective in most cases, as well as removing reliance on LLM service provider uptime, among other things. This gap in the existing research could become a much more critical one in the near future as the typically smaller, and therefore less resource-intensive, open-source LLMs become more and more sophisticated, allowing for deployment in ever smaller machines. Additionally, another research gap lies in the widespread usage of established privacy policy datasets such as the OPP-115 and MAPP datasets provided by the Usable Privacy Policy Project group [40][58]. This significantly narrows the scope of the research, especially with regards to newer changes in privacy policies that are made fairly often and aren’t reflected in these datasets, suggesting a need for evaluating LLMs on new, diverse datasets. Therefore, this thesis aims to investigate the applicability and adaptability of open-source LLMs for the task of privacy policy annotation on a dataset of new, real-world privacy policy texts. For this purpose, we aspire to design a system that can automatically perform the described evaluation.

3.2 Goals

As discussed, most of the current research relies on closed-source LLMs such as OpenAI’s GPT series of models, which comes with significant drawbacks, including high costs for the most sophisticated models, limited transparency in the models’ inner workings and potential privacy concerns as to how providers process potentially sensitive data. The usage of open-source models such as Meta’s Llama models, or Google’s Gemma models, alleviates some of these concerns. Despite the large range of open-source models available nowadays, this possibility has not been explored very much, with very little data on whether or not these models are able to match the accuracy and precision of their closed-source competitors. This evaluation has the goal of providing empirical insights into this topic, determining if open-source models currently are able to uphold the high standards required for legal document processing, whether or not they can serve as a viable, cost-effective alternative to established models, and if so, what

the best contender among the popular models is.

3.3 Objectives

From these general goals, we derive the following specific objectives:

Evaluating the feasibility of using open-source LLMs for privacy policy annotation

This will involve assessing their operational efficiency, including inference times, reliability, and ability to maintain the level of qualitative performance that OpenAI's models offer. However, feasibility here also refers to practical considerations such as computational requirements and adaptability to different types of privacy policies, for example multi-lingual ones. In fulfilling this objective, this project attempts to provide an empirically-based, comprehensive view of whether open-source LLMs can sustainably be used in real-world scenarios.

Finding the best-suited open-source LLM for privacy policy annotation

Since open-source language model offerings are a rapidly evolving and ever-changing field, this project aims to provide a tool that can be used to gain some form of insight into the suitability of models and, along with that, determine the best performer among the tested models that may be used in further research.

Providing a pipeline for the automatic evaluation of LLM performance

The third objective is to not only develop a pipeline that can be used to automatically annotate privacy policies, but also provide a tool for evaluating the outputs of the models. This tool should enable objective testing of LLM suitability for privacy policy annotation, further streamlining the process of future assessment of open-source LLMs and thereby benefiting the overarching goal of protecting users' online rights.

3.4 Requirements

Based on the previously outlined goals and objectives, several key requirements can be established to provide a framework for the design and evaluation of the system for privacy policy annotation using open-source LLMs. These requirements are derived from the necessity to ensure that the designed system meets the desired standards of performance, feasibility, and scalability. The requirements address technical, functional, and operational aspects, ensuring that the system fulfills the objectives effectively.

1. Accuracy and Precision in Annotation

One of the primary requirements is selecting LLMs that have a high degree of accuracy and precision in the annotation tasks of privacy policies. This involves the correct identification, extraction, and categorization of information such as data categories, processing purposes, user rights, and third-party data sharing practices as specified by GDPR and similar regulations. Accuracy is crucial here because incorrect annotations could lead to the misinterpretation of privacy policies, potentially negatively impacting the usability of the annotated data in the context of compliance checks. Precision here also means that the annotations should stick closely to the defined annotation guidelines without adding irrelevant information or missing key details, thereby ensuring that the annotations are both meaningful and actionable.

2. Robustness and Reliability of Model Outputs

The system must be robust and provide reliable outputs consistently across a wide range of privacy policies. Policy texts vary significantly in terms of language, structure, and complexity; therefore, the LLM must handle variations, as well as potential grammatical errors, for example, without a significant drop in performance. This includes dealing with legal jargon and things such as branded words/names. Reliability is also relevant with regards to the LLM's ability to minimize hallucinations and strictly follow the system prompts designed for annotation. The system should include mechanisms to handle these issues, where possible approaches could include model fine-tuning, the use of Retrieval Augmented Generation, or simply more prompt engineering, ensuring that the outputs remain within acceptable quality thresholds.

3. Scalability and Computational Efficiency

Given the potentially large scale of privacy policy documents that need to be processed, the system must be scalable. Of course, this is majorly dependent on the available hardware, which may vary between use cases. In general, though, this requirement emphasizes the need for the LLMs and the overall pipeline to efficiently process large amounts of requests. Computational efficiency is essential, particularly in an applied case where the volume of documents might be extremely large. The models should not only provide acceptable inference speeds, but do so with manageable computational costs, leveraging available GPUs effectively without excessive consumption or wasting of resources. This scalability also involves being able to handle updates in privacy policy formats or changes in annotation requirements without requiring significant re-engineering of the system.

4. Cost-effectiveness of the Annotation Process

One of the key differences between closed-source and open-source LLMs is the cost of inference. For closed-source LLMs available through APIs, this is typically dictated by the number of input tokens sent to the LLM, combined with the number of output tokens generated, with costs measured per token. Thus, open-source LLMs should provide a viable alternative to commercial models financially by significantly reducing cost compared to API usage costs. This should mainly stem from the fact that while inference time and, therefore, electricity consumption does increase with longer input and output, the combined mitigating effect of smaller models and lack of auxiliary costs apart from material cost and electrical bills should work out to a lower overall cost of inference per policy in most cases. The system should ideally be able to run on commonly available hardware configurations without the need for costly upgrades or cloud-based solutions, but also be compatible with these sorts of extensions.

5. Adaptability and Flexibility of the Pipeline

The pipeline used for evaluating the performance of LLMs must accommodate for different models and evolving requirements. This adaptability includes the ability to easily integrate newer open-source LLMs, update evaluation metrics, and modify annotation guidelines as privacy regulations and policy texts evolve. The pipeline should be modular, allowing for easy integration of additional components or extensions, such as data preprocessing, model inference, and output evaluation, to be adjusted independently without necessitating a redesign of the entire system. Flexibility also extends to the ability to incorporate feedback loops, enabling iterative improvements to the models based on evaluation outcomes and manual review processes, as well as taking into account the future possibility of incorporating fine-tuning or other

techniques.

6. Integration of Automatic Evaluation Metrics

To maintain a consistent and objective evaluation of LLM performance, the system must incorporate automatic evaluation metrics that accurately reflect the quality of annotations. These metrics should include traditional scores such as precision, recall, and F1-score, but also allow for the use of more specialized metrics that assess the alignment of annotated data with legal requirements, for instance. The evaluation framework should be capable of generating similarity scores between predicted and reference annotations ensuring a nuanced assessment compared to simply checking word-for-word similarity. This requirement ensures that the evaluation process is not only automated but also sufficiently sophisticated to capture the complexities within privacy policy texts.

7. Compatibility with Additional Automated Processing

Finally, the system must produce outputs that can be used in the scope of a larger pipeline, specifically the one developed by Thomas Cory that is further described in the next section. Compatibility and interoperability with other automated processes is key here, especially regarding things such as data extraction and document parsing. The outputs generated by the tool must be formatted consistently and structured in a way that downstream components can easily utilize, ensuring a seamless integration without the need for excessive, or ideally any, post-processing or reformatting. This includes adhering to standardized data formats such as JSON, and maintaining a clear hierarchy of annotated elements to enable further analysis. By ensuring compatibility with existing and future automated tools, the system should contribute to a comprehensive solution for privacy policy evaluation and eventually, compliance checking, enhancing overall efficiency and empowering not only users of online services, but also developers and privacy experts.

3.5 Methodology

Building upon these requirements, the system is constructed to cater to the objectives set earlier. The design for it is built upon an existing system by Thomas Cory and will be structured as depicted in Figure 3.1.

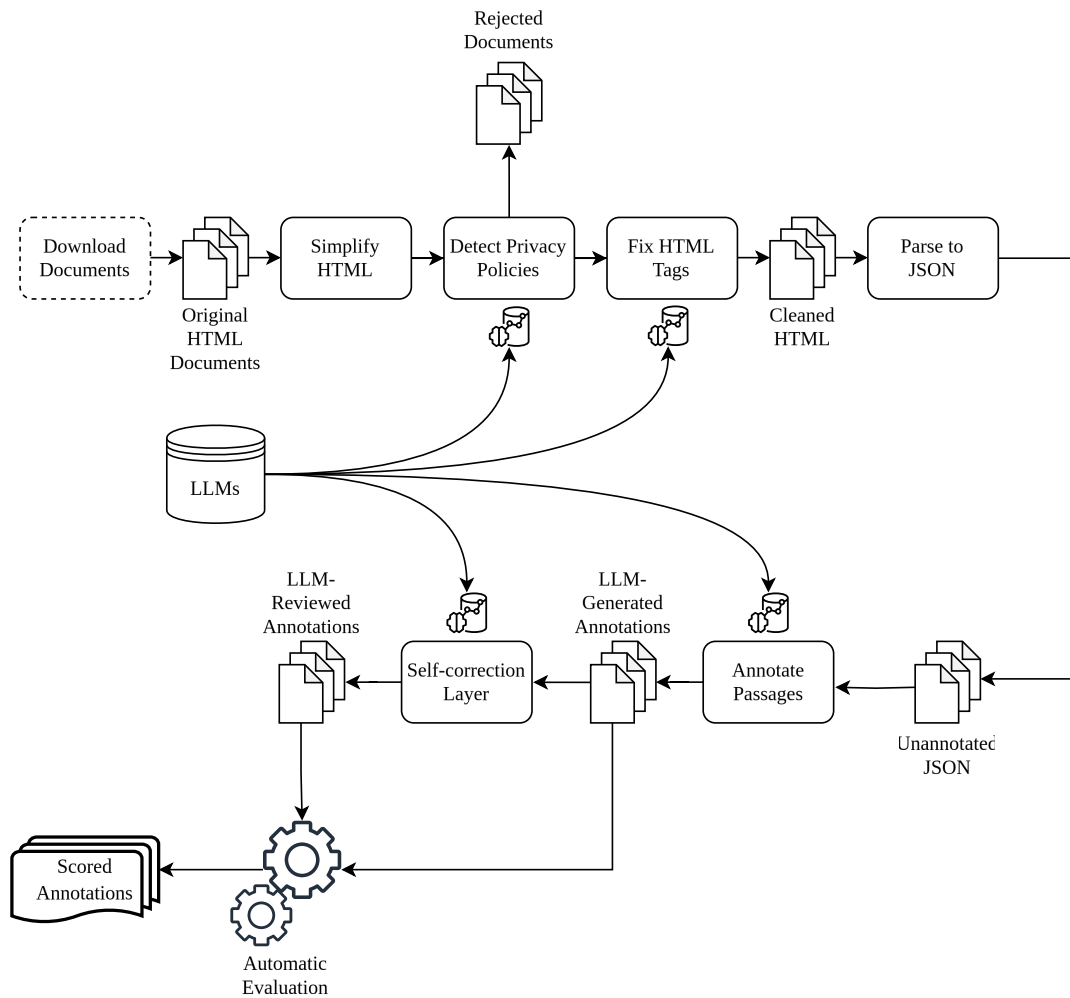


Figure 3.1: Project Pipeline

This thesis is going to focus on the use of LLMs within this pipeline and not the pipeline structure itself. LLMs are used in 3 separate stages of the pipeline: for detecting whether a given HTML document is a valid privacy policy, for identifying and fixing incorrect headline tag levels in the HTML structure and for the annotation itself. Previously, OpenAI’s GPT-4o was used in each of these steps. The aim of this project is to eliminate the dependence on closed-source LLMs which might not always be available via API due to unforeseen downtime of the provider, as well as privacy concerns regarding the use of our user input as training data for other closed-source models, for instance.

1. Gathering of Online Privacy Policies

Seeing as one of the objectives of this thesis is to provide an up-to-date analysis of modern privacy policy texts, these need to be gathered as a first step in the pipeline. Since apps offered on the Google Play Store are typically required to link to a privacy policy for app use, the Play Store can act as a form of policy registry where these types of documents can be found for almost all apps. The texts found here can easily be downloaded in their native HTML format for further processing.

2. Data Preprocessing

Since the original HTML files containing the policies aren't ideal for automated processing due to their inherent lack of uniform, cross-policy structure, some preprocessing must be done to filter out web pages that don't contain policies at all, such as error pages when sites are moved, as well as to store the policies in a more easily processable format such as JSON. To this end, four distinct steps are designed to produce an agreeable and clean format:

- a) A simplification step, during which the original complexity HTML is reduced, mainly for the next step,
- b) the detection step, where invalid pages are filtered out and discarded, and
- c) a fixing step during which HTML tags are replaced with more apt ones to preserve the semantic information they carry.

Making the policy texts more machine-readable in a preprocessing step alleviates some of the need for error and edge case handling during later pipeline steps. Semantics could also be considered during preprocessing, potentially adding important metadata for language models to take into account during annotation and review later.

3. Annotation and Review

The annotation step is the heart of the pipeline, where an LLM is tasked with extracting relevant information from a given policy and annotating it in accordance with the predefined guidelines. This step is the most precarious one, with it having the largest room for unforeseen errors, necessitated by the need to leave room for LLM creativity. During this stage, prompt engineering and iterative adjustments will become necessary, balancing strict guidelines and restrictions with the flexibility to explore additional metrics that may be unearthed by LLMs that introduce new aspects that weren't considered during the initial design process. The same applies to the self-correction step, during which a revision of the existing annotations is produced in order to emulate a "second opinion" approach that could also be achieved by incorporating a human-in-the loop process. Leveraging the existing framework for the original annotations in order to generate potential corrections and reasoning for the computed labels, thereby lending them additional reliability can be vastly beneficial in regards to the precision and accuracy of the resulting classifications made by the language models.

By following this design, the outlined concept aspires to provide a robust framework for using and evaluating open-source LLMs in the scope of privacy policy annotation. This design not only meets the established requirements but should also provide an efficient, modular and customizable solution for modern-day legal and compliance challenges.

3.6 Limitations of the Concept

While the proposed framework does address several clear research gaps, it nevertheless has its limitations that require consideration. Firstly, the reliance on newly gathered, unprocessed privacy policy datasets introduces variability, especially regarding formatting and consistency, which may complicate the pipeline itself as well as the evaluation process. Unlike established datasets such as OPP-115 and MAPP, which incorporate standardized formatting and manual checking by human experts, newly collected data may exhibit all kinds of inconsistencies. This variability can pose significant challenges in ensuring that the evaluation metrics accurately

reflect model performance across diverse policy documents. Additionally, the ever-changing nature of privacy regulations, which undergo frequent amendments and regional adaptations, may render the annotation guidelines and evaluation criteria rapidly outdated, thereby making frequent updates necessary in order to maintain the relevance and accuracy of the pipeline.

Another limitation that can not be neglected pertains to the general capabilities and performance of open-source LLMs compared to their closed-source counterparts. While open-source models offer advantages in terms of cost-effectiveness and transparency, they often lack the extensive fine-tuning and proprietary optimizations that make the performance of closed-source models like OpenAI's GPT series as coveted as they are. This discrepancy could result in open-source LLMs failing to capture some of the more nuanced information embedded within the analyzed policies. Consequently, the evaluation may reveal a performance gap that is not attributable solely to the evaluation framework itself but rather to fundamental differences in model architecture and training paradigms, adding another factor to consider during the assessment of the results. The scaling of the pipeline for large numbers of policies is also inherently limited by available hardware, making the use of this framework highly dependent on resources available to the user not only financially, but regarding the state of the global market, seeing as graphics cards are in the highest demand they have ever been [59][60] due to the widespread AI hype associated mainly with LLMs and diffusion models, driving their prices up and their availability down. Hence, performance reviewed during the evaluation of this project is entirely subjective and specific to the exact software versions and hardware used during the course of this work, making it only partially applicable to broader deployment scenarios making use of different hardware configurations and software environments in the future. This limitation emphasizes the necessity for continuous research in this field, as it remains a highly volatile and rapidly evolving landscape. The findings of this project might be entirely overturned by a new LLM just weeks after completion, consequently making a new evaluation necessary.

A final possible source of errors is the project's reliance on automated evaluation methodologies. It introduces the potential for missing subtleties and contextual interpretations that human evaluators might naturally identify. Automated metrics, while essential for scalability and efficiency, might not fully capture the intricate legal language, implicit meanings, and intrinsic contextual dependencies within privacy policies. This limitation could result in certain oversights that a human would typically recognize, thereby affecting the overall accuracy and reliability of the evaluation metrics. For instance, when annotating a passage within a policy document, an LLM might interpret a larger portion of the sentence as pertinent to a certain privacy requirement than the benchmark model. This can be acceptable if the meaning of the identified passage extract is not altered by including more words than the gold standard does, but can quickly introduce ambiguities that are nigh impossible to accurately take into account during the evaluation of the pipeline results due to limited context and the need for definitive and deterministic evaluation measures. The focus of this project on a fully automated and scalable pipeline rules out the possibility of manually checking all of the annotations, while simultaneously eliminating the use of LLMs in another evaluation step, as their partial reliance on randomness makes them unsuitable in the scope of such an evaluation. Therefore, this limitation is unavoidable in some form, though it should be somewhat mitigated by even distribution within the large sample size. Future research, however, could explore hybrid evaluation frameworks that incorporate periodic human-in-the-loop assessments to supplement

automated processes, thereby enhancing the overall reliability and accuracy of the annotation system.

4 Implementation

Following the definition of project requirements and objectives that are to be fulfilled in chapter 3, this now poses the question of *how*. As there are quite a number of different components within the pipeline stages that rely on shared libraries and frameworks, it is beneficial to first provide an overview over the main packages used and their purposes. This information will then serve as a foundation for the further comprehension of the pipeline. The pipeline is written entirely in Python and relies on a few key libraries that, as is often the case, utilize pre-compiled C code in the background.

4.1 Dependencies

Ollama

Ollama [61], at its core, is a platform that provides both a server framework that can efficiently run LLMs and a corresponding HTTP API for sending requests. Ollama also provides an extensive library of open-source LLMs, all of which are contributed to by community members and easily accessible via the API. The main benefits of using the Ollama platform for local LLM deployment stem mainly from the built-in performance enhancements and its ease of use. The service natively supports multi-GPU hardware configurations and enables the execution of LLMs even with limited RAM and VRAM, while also providing a simple interface in the API for developers to take manual and granular control over LLM generation settings such as adjusting model temperature, setting maximum token output limits and more. This is significantly advantageous to the usability of the overall system since most current LLMs are optimized for use with CUDA on NVIDIA graphics cards. Developing a system without Ollama or a similar service that offers a comparable amount of abstraction would either severely limit its adaptability to different hardware configurations or require extensive additional effort to implementing this from scratch. Furthermore, Ollama's management of model weight distribution across multiple GPUs and RAM/VRAM vastly promotes the cross-platform usability of the system on a multitude of hardware configurations, making the system much more adaptable, portable and scalable. This combination of native efficient resource management, performance enhancements, ease of use and reliability makes it the perfect choice for this project, as the additional customizability that the Hugging Face Transformers library [62] offers, for instance, is not essential for the functioning of the system at this stage. For these reasons, Ollama was chosen as the framework for running LLMs locally within this thesis.

Gensim

The Gensim Python package [63] is a framework designed for a number of natural language modeling tasks, the key task for this project being word vector embeddings. More specifically, one of the embedding algorithms implemented by the package is the fastText algorithm, first

published to GitHub by Facebook’s AI Research lab in 2016 [64]. The efficient vector embeddings generated using Gensim can capture semantic and, by utilizing the fastText algorithm, subword information, enabling an in-depth evaluation of phrases and sentences that does not just rely on blunt syntax comparisons. Gensim also offers alternative models and comprehensive documentation for similar tasks which allows for potential expansion on its functionality in the future, making it ideal for the project. The library additionally enables the training of custom embedding models on a proprietary dataset as well as continuing training a pretrained model (i.e., fine-tuning it), which is a proven method for improving the accuracy of the embeddings. Acquiring pretrained fastText models is also reasonably easy, though the differences between distinct models offered in various places can be hard to spot at a first glance. Gensim itself provides a downloader API that offers one set of models, while some of the same models are downloadable directly from the fastText website, which also contains some unique models that are not accessible via the Gensim API. While this does introduce some obscurity, it also means that embedding models are not too scarce. Another advantage of the package is that it is optimized for use with large corpora of text, as well as for less powerful hardware, further contributing to our system’s overall efficiency and scalability. Gensim is used for the automated evaluation and scoring of annotations generated using the pipeline and, as such, is a vital part of the project.

NLTK

NLTK, as discussed in chapter 2, is a Python library used for a wide variety of NLP tasks. Its role in this project is fairly simple: it is used for the creation of a custom dataset that the vector embedding model used for the evaluation is fine-tuned/trained on. One of the features of the package is a lightweight tokenizer that can be used to split large paragraphs of continuous text into sentences, which is necessary since the embedding models require training data in the form of tokenized sentences.

RapidFuzz

Another auxiliary language analysis library used in the scope of the evaluation step is RapidFuzz [65]. This package is built upon an older version of the popular fuzzy string matching library FuzzyWuzzy [66] and enhances the base functionality of FuzzyWuzzy by implementing a large portion of it in C++. Both libraries use the Levenshtein distance [67] to calculate string similarity on a very basic level, a metric which is used in this thesis to augment the semantic similarity scores calculated for the annotations since the accuracy of those can vary with the amount of context given.

BeautifulSoup

This widespread Python package is used in the pipeline to manipulate, restructure, and clean up HTML files containing privacy policies. Its extensive functionality and expansive documentation make it an easy library to work with while maintaining a readable code structure even when dealing with HTML, which is notoriously difficult for humans to parse in its raw form.

Selenium

Finally, the Selenium package is also used to process web pages. However, unlike the BeautifulSoup library, Selenium, at the heart of which lies the Selenium WebDriver, is used to procure the privacy policies investigated in this thesis. While BeautifulSoup’s primary use case is manipulating static HTML and XML files, Selenium excels in web browser automation, where

navigating between pages and following links may be needed, for instance. Selenium is also much more useful for handling websites that heavily incorporate JavaScript and, as such, is a more suited tool for navigating to an app's privacy policy page.

4.2 Prerequisites

For this thesis, the pipeline was run on an Ubuntu Virtual Machine (VM) on a server with access to a 4-core CPU, 100 GB of RAM, and 2 NVIDIA RTX 3090 graphics cards, which have 24 GB of VRAM each. This sets some constraints for model size: the total amount of VRAM needed to run inference completely on the GPUs not only consists of the amount of memory needed to load the model weights but is also increased by the amount of context used in each query. Large models, such as Meta's Llama 3.1 405B model, need too much memory and can not be considered for this project. With these restrictions in mind, the open-source LLMs listed in Table 4.1 were settled on for the evaluation.

Model Provider	Model Name and Specifications	Ollama Model Code
Meta	Llama 3 8B	llama3:instruct
Meta	Llama 3.1 8B	llama3.1:8b-instruct-q4_0
Meta	Llama 3.1 8B with F16 quantization	llama3.1:8b-instruct-fp16
Google	Gemma 2 9B	gemma2:9b-instruct-q4_0
Google	Gemma 2 27B	gemma2:27b-instruct-q4_0
Mistral AI	Mistral Nemo 12B	mistral-nemo:12b-instruct-2407-q4_0
Mistral AI	Mixtral 8x7B	mixtral:instruct
Microsoft	Phi 3 14B with 128k context window	phi3:14b-medium-128k-instruct-q4_0
Alibaba Group	Qwen2 7B with F16 quantization	qwen2:7b-instruct-fp16

Table 4.1: LLMs used in the Project

Since models offered on Ollama, especially those published by larger organizations, receive updates occasionally, it is essential to note that these models' weights were last updated on September 6, 2024.

As evidenced by the Ollama model codes listed in the table, the models used in the analysis are not the standard versions of the models but the instruct-finetuned versions. These models undergo additional training on top of the regular models, which is designed to follow human instructions more accurately and effectively. Unlike the ordinary versions of the LLMs, which are typically trained on vast amounts of general data to output human-like text, instruct-finetuned models are refined using methods such as the commonly used Reinforcement Learning from Human Feedback (RLHF). During this process, the models are trained on a set of specific task instructions and their desired outputs, which helps them understand and execute complex commands more effectively and adhere more closely to guidelines, traits that are incredibly beneficial to this project. Given that close following of the given system prompts

is critical for the annotations made by the LLMs to be actionable, the use of instruct-finetuned models is ideal for this thesis. Furthermore, LLM output is always subject to some randomness. While this is central to the nature of these models and part of what makes them so useful, it also introduces some uncertainty regarding the consistency of the output. The attention mechanisms used in LLMs attempt to resolve an issue that has plagued language models for years: weighing the significance of different sub-sequences within the input. Large Language Models excel at keeping track of essential phrases and words within inputs. However, the inputs used for this project can be extremely long, making it difficult for even modern models to keep track of details in their instructions. For example, some LLMs tend to output additional conversational text along with the desired output, which is detrimental to the pipeline as an exact output format is expected to function without hiccups. Leading and trailing text, as well as inaccurately placed brackets, quotation marks, or commas, can interrupt the pipeline, and while LLM output postprocessing can be done, more random or unexpected outputs can still break the workflow, making it critical to get as clean and strictly formatted output as possible in the first place to reduce potential points of failure. These are the reasons why instruct-finetuned models were chosen for the project, as they offer the best balance between adherence to instructions, consistency, and reliability, ensuring the outputs align precisely with the strict formatting requirements of the pipeline.

An additional noteworthy detail in the table above is that some models have specified quantizations. Quantization is a technique widely used for model size reduction in the realm of machine learning and it involves reducing the precision of the floating point numbers used to represent the model. By doing so, model sizes are vastly decreased, saving disk space, improving model loading times, inference times and memory requirements. However, this comes at the cost of accuracy and stability. The majority of models available from Ollama are Q4 quantized by default, while typically being offered in a wide variety of other quantizations, the most precise being F16 (or FP16) quantization. In order to take these quantized models into account as well within the frame of the project, Meta's Llama 3.1 8B model was evaluated for both quantizations, while Qwen2 7B was evaluated only for the F16 quantized version.

Another vital aspect of using LLMs is that their output can be influenced by tweaking model parameters. These parameters have distinct effects on the text generated by the models, a selection of which are:

- **Temperature:** controls how "creative" the LLM's response is by increasing or decreasing the randomness of the next chosen token
- **Top p:** when LLMs determine the next token to output, they consider a certain amount of possible tokens. The top p-value denotes the threshold of probability a token has to cross to be considered as a valid output token, meaning a higher top p-value makes the output less improbable
- **Top k:** controls how many tokens are considered for the next output token in every inference step. For instance, a value of 4 would mean that only the top 4 most likely tokens are considered in each step
- **Context window:** how many preceding tokens are considered as context in each step. For most modern LLMs, the maximum context window is 128 000 tokens

For this project, the chosen parameters are:

- **Temperature:** 0.5
- **Top p:** 0.9
- **Top k:** 40
- **Context window:** 8192 for smaller models, 6144 for larger models ($\geq 20\text{B}$ parameters)

These parameters were chosen to guide the models in adhering more closely to the system prompt and the existing wording in the prompt. The context window reduction for larger models proved necessary on the given hardware since the large context window necessitated for the evaluation leads to significantly higher VRAM usage than during regular use. Though a model's VRAM usage is not solely correlated with its parameter count, it is a good indicator for minimum usage and was sufficient for this task. It is also vital to note that the value ranges for these parameters are specific to Ollama and that while the maximum temperature range for models run using Ollama, for example, is between 0 and 1, OpenAI defines a range limit of 0 - 2.

Lastly, a choice was made on the number of policies to evaluate and the exact policies to consider. To achieve a balance between the prospective quality of the selected policies without prior analysis and being representative of the overall landscape of policies available to us via the Google Play Store, 25 policies were chosen at random from the Play Store's top 1000 apps, sorted by volume of downloads, and 25 others randomly from the rest of the apps. The top 1000 apps all have over 10 million downloads. This ensures that policies are chosen not only from apps that likely have a valid and comprehensive policy but also paints a picture of the rest of the apps, which may not have policies at all or have shorter and less legally sound ones. This leaves us with a dataset of 50 policies evaluated by the models named in Table 4.1.

4.3 Pipeline

The pipeline is divided, as discussed in Chapter 3, into sequential stages, or phases, that automate the whole process from acquiring the concrete policy texts and annotating them up to reviewing the annotations. Each step of this workflow aims to address unique challenges, such as unified formatting and LLM inaccuracies, to enhance the annotation quality and reliability. In the following, these stages are described in detail.

4.3.1 Accumulation of Privacy Policies

During the first step of the pipeline, the concrete policy documents to be annotated are gathered. This step can be skipped should the policies have been downloaded manually beforehand or already scraped during a previous evaluation. However, if this is not the case, the corresponding Google Play Store page of the specified package (or packages) is visited. On an app's Play Store web page, there is a section for app support under which the developer must provide a link to the app's privacy policy. Using the Selenium WebDriver, this page is then navigated to and downloaded. The result of this pipeline stage is a collection of raw HTML files containing the policy text and the surrounding HTML structure and styling.

4.3.2 Data Preprocessing

Since the raw HTML files have limited value by themselves due to their often convoluted structure, some processing is done in order to standardize the format of the policy texts. In the first step, the HTML structure of the policy documents is cleaned, removing unnecessary structural HTML tags, splitting text portions separated by `<br>` tags into separate lines and flattening the overall structure to result in an HTML-like document in which individual passages of the policy text are separated into unique lines, are accompanied only by their original containing structural HTML tag (i.e., `<h2>`, `<p>`, etc.) and are not nested within other structures. This vastly simplifies the parsing of the documents in subsequent steps. In the next preprocessing step, an excerpt from the cleaned HTML document is taken from the approximate middle of the file and fed into the LLM used for the pipeline.

Next, the LLM is instructed to discern whether the given excerpt is likely to be part of a privacy policy document content-wise. This is necessary as there are occasionally cases where the website linked to on an app's Play Store page does not contain a valid privacy policy. The reason can range from simply outdated links and moved pages to temporary connection errors or similar issues. Filtering these invalid documents out before the main stage of the pipeline prevents unnecessary resource consumption and streamlines the process overall.

The subsequent stage in the pipeline tackles a less syntactic issue: Since later phases of the pipeline interpret the remaining HTML tags in the document not only as semantic information concerning the importance of the contained text but also as context for passages following it, it is moderately important that the containing tags of the passages accurately reflect the relationships of different passages to each other. For example, one passage might be a table header or a list title and be formatted as such in HTML. However, due to the flattening of the structure done during previous processing, the hierarchical relationship between list items and header might be lost. By taking the HTML tag associated with a passage into account, this kind of relationship can be remodeled, in a way, lending more context to each passage. For this to work, though, the used HTML tags must accurately reflect the meaning of a passage within the document, which is frequently not the case, as modern web pages typically resort to using `div`-tags for most content and handling formatting through other tools, resulting in an HTML structure that contains virtually no semantic information within its tags. To fix this, our system employs an LLM analysis step during which the LLM attempts to infer correct HTML tags for passages that may need improvement. This results in an HTML file with a flat structure, where each passage of the policy contained in the page is wrapped in a meaningful and accurate tag.

Finally, the HTML file is then parsed into a list of objects in JSON format, where each entry in the list of JSON objects is formatted as defined by the JSON schema in Listing 4.1.

Listing 4.1: Unannotated JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "description": "",
  "type": "object",
  "properties": {
    "type": {
      "type": "string"
    },
    "context": {
      "type": "array",
      "uniqueItems": true
      "items": {
        "required": [
          "text",
          "type"
        ],
        "properties": {
          "text": {
            "type": "string"
          },
          "type": {
            "type": "string"
          }
        }
      }
    },
    "passage": {
      "type": "string"
    }
  },
  "required": [
    "type",
    "context",
    "passage"
  ]
}
```

A detailed example can be found in Appendix 6, Section 1, Listing 1. Each object has an associated "type" entry that denotes the HTML tag that encapsulates the passage. It also contains a "context" list in which a simplified version of the surrounding elements is saved. Lastly, it includes the passage itself. This format accomplishes multiple things: not only does it break the policy up into more easily processable pieces, but it also ensures that each passage can still be viewed in the context of its surrounding original structure without having to evaluate the entire file at once. This is hugely beneficial to the further analysis of the document, as context size for LLM input has limited benefits when growing too large. Overly long inputs lead to drastically inflated hardware loads and decreased accuracy, making splitting the policies into smaller contextualized portions essential for the system's performance. The completion of this step marks the end of the preprocessing phase, leading into the annotation phase.

4.3.3 Annotation

During the annotation of the policies, it is vital that the LLM used makes functional annotations that adhere to specific guidelines that ensure the legal actionability of the annotations, as described in the project requirements. Therefore, we specify a number of concrete transparency requirements, taken from the GDPR, that the LLM is instructed to annotate. These are defined as follows:

1. **Controller Name:** The name of the data controller.
2. **Controller Contact:** How to contact the data controller.
3. **DPO Contact:** How to contact the data protection officer.
4. **Data Categories:** What categories of data are processed.
5. **Processing Purpose:** Why is the data processed.
6. **Source of Data:** Where does the data come from, e.g., third parties or public sources.
7. **Data Storage Period:** How long is the data stored.
8. **Legal Basis for Processing:** What is the legal basis for the processing.
9. **Legitimate Interests for Processing:** What legitimate interests are pursued by processing the data.
10. **Data Recipients:** With whom is the data shared.
11. **Third-country Transfers:** Is the data transferred to a third country, and what safeguards are in place.
12. **Automated Decision Making:** Is the processing used for automated decision making, including profiling.
13. **Right to Access:** The user has the right to access their personal data.
14. **Right to Rectification:** The user has the right to correct their personal data.
15. **Right to Erasure:** The user has the right to delete their personal data.
16. **Right to Restrict:** The user has the right to restrict processing of their personal data.
17. **Right to Object:** The user has the right to object to processing of their personal data.
18. **Right to Portability:** The user has the right to receive their personal data in a structured, commonly used, and machine-readable format.
19. **Right to Withdraw Consent:** The user has the right to withdraw consent to the processing of their personal data.
20. **Right to Lodge Complaint:** The user has the right to lodge a complaint with a supervisory authority.

Providing the LLM with these fixed, predefined categories for the annotation guarantees a base level of reliability and delivers a framework of privacy requirements to work within, as compared to letting the LLMs generate their own categories, which would vary from run to run and between models. The specified transparency requirements are embedded in the model's system prompt, making them a constant for every query during the annotation phase. Equipped with this system prompt, the model then evaluates each segment of the given pri-

vacy policies in succession, where each segment, or passage, is a JSON object as described above, generating a list of annotations that it attaches to the passage object. An example of this annotation list might look like the example in Appendix 6, Section 2, Listing 2 and should follow the JSON schema shown in Listing 4.2.

Listing 4.2: Annotated JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "type": {
      "type": "string"
    },
    "context": {
      "type": "array",
      "uniqueItems": true,
      "items": {
        "required": [
          "text",
          "type"
        ],
        "properties": {
          "text": {
            "type": "string"
          },
          "type": {
            "type": "string"
          }
        }
      }
    },
    "passage": {
      "type": "string"
    },
    "annotations": {
      "type": "array",
      "uniqueItems": true,
      "items": {
        "required": [
          "requirement",
          "value",
          "generalized_value",
          "performed"
        ],
        "properties": {
          "requirement": {
            "type": "string"
          },
          "value": {
            "type": "string"
          },
          "generalized_value": {
            "type": "string"
          },
          "performed": {
```

```

        "type": "boolean"
    }
}
},
"required": [
    "type",
    "context",
    "passage",
    "annotations"
]
}

```

The "annotations" entry generated by the LLM is added to the original passage as an additional key in the object. Each annotation is required to include a key denoting the transparency requirement referenced in the annotated portion of the passage, the precise segment of the passage pertinent to the identified requirement, a generalized value that is freely chosen by the LLM, and a "performed" value that is either true or false and signifies whether the identified requirement is met by the annotated section. The LLM is required to output the annotations list *and* the entire input object, making postprocessing obsolete if this is done precisely according to its guidelines. However, the reason for this is not to avoid postprocessing; quite the contrary, having the LLM output only the annotation list would be a noticeable improvement to the efficiency of the pipeline and reduce computational load. Requiring the LLM to output the complete input object before providing its annotation list serves an essential objective: fostering a "train-of-thought" inference process and ensuring the model's annotations are directly linked to the exact input. Since LLM outputs are generated sequentially, with each token influenced by the preceding ones, this helps solidify the input text as the foundation for the subsequent annotations. It also reduces the risk of the LLM generating an incorrect token as its first token and basing the rest of its output on that faulty token or word, a phenomenon often observed when LLMs provide either "yes" or "no" as the first word in their response. By providing its own context first, the model reinforces its contextual understanding of the original input before generating the annotations, minimizing the risk of hallucinations and false or poorly aligned outputs. This method, the implicit "re-reading" of the LLM's own output, is a widespread strategy for guiding them toward more reliable outputs and ensuring that the annotations made in the scope of this project precisely reflect the intended content, reducing divergences that could undermine the integrity of the annotations.

Ollama's support for parallel requests further enhances the annotation process by allowing multiple passages to be processed simultaneously, improving throughput and reducing the overall runtime of the system, especially for large-scale annotation tasks. This parallelization leverages asynchronous programming and a custom request client provided by Ollama to handle multiple queries concurrently. This approach not only optimizes the efficiency of the annotation phase but also provides a speedup in the subsequent reviewing process, where the same parallelization technique is employed to improve inference times, as described in the following section.

This annotation approach ensures that each passage is systematically evaluated against consistent criteria, ensuring sufficient quality and comparability of the annotations. By defining explicit requirements and concrete descriptions for each category's content, the annotation

process becomes both repeatable and interpretable, reducing ambiguity and providing clear insights into the extent of compliance with privacy regulations. This annotation framework not only reinforces the integrity of the annotations but also lays a dependable foundation for subsequent stages of the project.

4.3.4 Reviewing

In the last step of the pipeline, an effort is made to mitigate hallucinations and erroneous annotations, as well as formatting errors that arise even with all of the discussed optimizations to prevent this. For this purpose, the same LLM used throughout all previous steps of the pipeline is prompted with a passage object obtained through the previous annotation phase and tasked with evaluating it, providing a review of the annotations made for the given passage, scoring it, and generating a corrected or improved version of the annotations. As during the annotation phase, the model is instructed to output the entire input along with its output. This, again, serves the purpose of improving the accuracy of the LLM's output by forcing it to generate a "train-of-thought" type output. Compelling the model to reason before making corrections to the given annotations improves the output consistency, compared to the model potentially providing incorrect revisions and subsequently justifying them in its review since it cannot backtrack on and correct tokens it has already generated. The review of the annotations that the LLMs generate also helps human reviewers understand and take reasoning into account for the LLM's corrections in the future. An example of an LLM-generated review of the examples provided in the preceding sections can be found in Appendix 6, Section 3, Listing 3. The corresponding JSON schema can be found in Listing 4.3.

Listing 4.3: Reviewed JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "review": {
      "type": "string"
    },
    "score": {
      "type": "number"
    },
    "revised": {
      "type": "object",
      "properties": {
        "type": {
          "type": "string"
        }
      },
      "context": {
        "type": "array",
        "uniqueItems": true,
        "items": {
          "required": [
            "text",
            "type"
          ],
          "properties": {
```

```

        "text": {
            "type": "string"
        },
        "type": {
            "type": "string"
        }
    }
},
"passage": {
    "type": "string"
},
"annotations": {
    "type": "array",
    "uniqueItems": true,
    "items": {
        "required": [
            "requirement",
            "value",
            "generalized_value",
            "performed"
        ],
        "properties": {
            "requirement": {
                "type": "string"
            },
            "value": {
                "type": "string"
            },
            "generalized_value": {
                "type": "string"
            },
            "performed": {
                "type": "boolean"
            }
        }
    }
},
"required": [
    "type",
    "context",
    "passage",
    "annotations"
]
},
"required": [
    "review",
    "score",
    "revised"
]
}

```

4.4 Limitations

Despite all of the previously detailed efforts to achieve as sophisticated a pipeline as possible, it does have its limitations. The limitations of this implementation of the pipeline can be categorized into two overarching categories: hardware limitations and software limitations. The software limitations, in turn, can be divided into limitations of code developed in the scope of this thesis and limitations of proprietary software.

4.4.1 Hardware Limitations

While the NVIDIA GeForce RTX 3090 graphics cards used for this project are excellent consumer GPUs, their performance is not comparable to high-end tensor core cards specialized for handling LLMs, such as the NVIDIA A-series or H-series of GPUs. This is a central point of improvement, as the performance of LLMs heavily depends on the computational resources available, especially regarding available VRAM in addition to raw processing power. Limited resources lead to longer inference times, impacting the pipeline's efficiency and forcing the use of smaller, less capable models that may lack the accuracy of larger, state-of-the-art LLMs. Insufficient memory, particularly GPU memory, can lead to models being offloaded to system RAM by Ollama, which can be helpful for simple tasks since it enables model running even exceeding the theoretical limits of the installed GPUs. For this project, however, this is detrimental to the performance of the pipeline as it drastically increases inference time to the point of impracticality and is the reason why the context window of the queries was reduced when using larger models. More VRAM would be critical in increasing the system's efficiency by allowing for more concurrent requests and the use of larger models or ones with higher precision quantization. As evidenced by the scenario, though, most of the constraints stem from intertwining hardware *and* software limitations and the need to balance both against each other.

4.4.2 Software Limitations

The software limitations (though, as discussed, the line between both types of limitations is slightly blurred because hardware limitations can be alleviated through software and vice versa) can be further separated into those related to code developed within the scope of this project and those inherent to the proprietary software used in the system.

The custom code written for this system contains the most potential for error. Mainly, the system prompts used at various stages of the pipeline can and should be continuously optimized. There cannot be a single perfect system prompt for each task, not only because every model is different structurally and takes better to different formats, but also due to the inherent complexity of natural language that results in a near infinite combination of interpretations, nuances, and variations that are impossible to distill into a single optimal prompt. Therefore, while the system prompts used produce good results with the chosen LLMs, there is always room for improvement here.

An adjacent topic is flawed language model outputs. Here, errors might stem from the models outputting unexpected tokens, such as additional leading and trailing text accompanying JSON outputs or incorrectly formatted JSON objects with missing commas and brackets. Once again, the inherent randomness in LLM inference is both a blessing and a curse, allowing for

the exploration of new and innovative annotation possibilities while maintaining a base level of uncertainty that cannot be eliminated completely.

Something else the code currently does not take into account is the amount of VRAM used by the models. Since this is a hardware limitation, a choice for the default context window was made for each step of the pipeline and used for all of the models, regardless of their actual GPU memory usage. By not adjusting this dynamically, efficiency is potentially lost.

Moreover, a quickly overlooked but essential step in the pipeline is the detection of valid policies. This is currently done by inputting an excerpt of a cleaned HTML file, taken from a fixed position around the middle of the document, into an LLM and prompting it to determine whether or not the content could belong to a privacy policy. For most policies, using the entire HTML file as input for the detection stage is not feasible due to hardware limitations, which is precisely the reason for the current approach. However, there are inevitably policies that slip through the system's net with this procedure, an example of which could be a site that has a long list at the beginning of the policy that contains text not easily identified as pertinent to privacy without further context which is then used for the detection. Given modern policies' typical lengths, an enormous amount of GPU memory would be required to use complete policies for detection reliably across a large sample size. Therefore, if this hardware requirement cannot be realized, a different approach to the detection of privacy policy validity would be necessary.

Having already discussed the usage of Ollama briefly, this platform introduces its own limitations to the project. Primarily, the system's dependence on Ollama makes it subject to any changes made by the maintainers of the open-source software and potential vulnerabilities introduced to it. It also does not allow as granular control over the LLMs as the HuggingFace Transformers library does, for instance, which could be a safer bet for future-proofing. On top of the dependence that it introduces, Ollama also does not natively support the finetuning of models, which, while not making it impossible, complicates it should it become desired in the future.

5 Evaluation

To evaluate the suitability of the chosen LLMs for legal document analysis, their annotations are evaluated against a dataset of annotations made by OpenAI’s GPT-4o, which is an advanced, albeit closed-source, model that has proven to produce highly reliable and excellent quality annotations without any additional changes, making it an ideal benchmarking LLM at the time of writing. Established evaluation metrics such as precision, recall, and F1-score, provide a basis on which we can analyze the contender models’ performance and suitability in an objective fashion that is potentially even comparable across projects. The development of a robust evaluation framework ensures that new LLMs can be continually assessed as they emerge in the rapidly developing landscape of open-source language models.

Given the innate variability in LLM-generated texts and the impracticality of manual evaluation due to its time-consuming and subjective nature, an automated evaluation that leverages semantic similarity as an evaluation metric is ideal. This way, annotations can be objectively compared based on their meaning rather than exact wording, providing a level of comparability that would be impossible due to natural human bias and error unavoidable in a manual evaluation process. The semantic similarity scores necessary for this can be calculated using vector representations of words and phrases, specifically by using vector embedding models like FastText, or its predecessor, Word2Vec. This method offers consistent and reliable scoring, accommodating phrasing variations, paraphrasing and varying granularity of annotations that invariably occur during the automated annotation phase of the pipeline. Additionally, FastText models are computationally efficient and can be finetuned unsupervised on our dataset of privacy policies, enhancing their ability to capture the technical and legal language used therein. This approach balances the need for accuracy with practical considerations of computational resources, providing a feasible and effective means of evaluating LLM performance.

5.1 Methodology

The evaluation pipeline can be subdivided into 4 primary phases: the finetuning of the used FastText model, a short data preprocessing stage, an annotation comparison stage during which the annotations made by the reference model are compared to those made by the contender model that is being evaluated, and the calculation of the metrics themselves.

5.1.1 Embedding Model Finetuning

Before the evaluation itself, the FastText model used, an embedding model trained on Wikipedia texts and incorporating subword information (available from the FastText site [68]), is finetuned on the complete set of 50 policies that was used for the original annotation process. This is achieved using the Gensim package and results in an embedding model file that can be loaded

for the subsequent evaluation.

5.1.2 Data Preprocessing

The evaluation process involves three datasets for each model:

- (1) **Reference Annotations:** A set of annotations made by OpenAI’s GPT-4o that serves as the ground truth for the evaluation.
- (2) **LLM-Generated Annotations:** Annotations produced by the LLM under evaluation.
- (3) **LLM-Reviewed Annotations:** Reviewed versions of the annotations made by the contender LLM.

To ensure a fair comparison, these datasets undergo some simple preprocessing to normalize the text, such as converting to lowercase and stripping whitespaces.

5.1.3 Annotation Comparison

Before comparing annotations, a consistent score must be defined that can be used to assess the similarity of annotations. Since FastText only provides vector representation of words and phrases, and not a concrete score, this must be defined as well.

5.1.3.1 FastText Embeddings

FastText only supports the embedding of single words at a time. The conventional strategy for calculating phrase vectors is as follows:

- (1) **Word Embeddings:** Each word in a phrase is converted into a vector representation using the FastText model.
- (2) **Phrase Embeddings:** The embeddings of individual words in a phrase are averaged to obtain a single vector representing the entire phrase:

$$\vec{v}_{\text{phrase}} = \frac{1}{n} \sum_{i=1}^n \vec{v}_{\text{word}_i} \quad (5.1)$$

where n is the number of words in the phrase.

5.1.3.2 Cosine Similarity

The similarity between two phrases is computed using cosine similarity of their embeddings:

$$\text{Similarity}(\vec{v}_1, \vec{v}_2) = \frac{\vec{v}_1 \cdot \vec{v}_2}{\|\vec{v}_1\| \|\vec{v}_2\|} \quad (5.2)$$

This metric measures the cosine of the angle between the two vectors, providing a value between -1 and 1 , where 1 indicates identical orientation, which corresponds to maximum similarity.

5.1.3.3 Optimal Matching Algorithm

A cost matrix C is constructed to represent the dissimilarity between each pair of reference and LLM-generated annotations:

$$C_{ij} = \begin{cases} -\text{Similarity}(\text{Ref}_i, \text{Cont}_j), & \text{if Similarity} \geq \tau \\ H, & \text{otherwise} \end{cases} \quad (5.3)$$

where:

- Ref_i is the i -th reference annotation.
- Cont_j is the j -th contender LLM-generated annotation.
- τ is the similarity threshold. This was chosen as 0.8 for this evaluation.
- H is a high cost value representing disallowed matches.

The optimal matching is then found using the Hungarian algorithm, which minimizes the total cost:

$$\text{Minimize } \sum_{(i,j) \in M} C_{ij} \quad (5.4)$$

where M is the set of matched annotation pairs.

5.1.4 Metrics Calculation

Additionally to calculating a semantic similarity score as described above for every matched annotation and saving the complete list of scores, we calculate a word-level similarity score for every matched annotation using the RapidFuzz package discussed earlier, specifically, the `rapidfuzz.fuzz.ratio()` method that leverages the Levenshtein distance to assess the similarity of two strings. These values are also saved. Furthermore, for each model, we calculate the evaluation metrics described in the subsequent subsections.

5.1.4.1 True Positives (TP), False Positives (FP), and False Negatives (FN)

- **True Positives (TP):** Matched annotation pairs where the LLM-generated annotation correctly corresponds to a reference annotation.
- **False Positives (FP):** LLM-generated annotations that do not match any reference annotation.
- **False Negatives (FN):** Reference annotations that do not have a corresponding LLM-generated annotation.

5.1.4.2 Correct Requirements Ratio

The correct requirements ratio measures the proportion of requirements correctly identified by the LLM:

$$\text{Correct Requirements Ratio} = \frac{|\text{Reference Requirements} \cap \text{Contender Requirements}|}{|\text{Reference Requirements}|} \quad (5.5)$$

Both of these are sets, meaning that this metric calculates the proportion of the distinct requirements occurring in a passage that were correctly identified by the reference model.

5.1.4.3 Precision, Recall, and F1 Score

These three widespread metrics for classifier evaluation are calculated twice. First, the metrics are calculated counting every individual correctly matched annotation as a true positive, every reference annotation that the contender did not make as a false negative and every contender annotation that was not found in the reference annotations as a false positive. Then, the metrics are calculated again, this time only counting **distinct** requirements that were annotated instead of entire annotations, regardless of how often they were annotated.

- **Precision:**

$$\text{Precision} = \frac{TP}{TP + FP} \quad (5.6)$$

- **Recall:**

$$\text{Recall} = \frac{TP}{TP + FN} \quad (5.7)$$

- **F1 Score:**

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5.8)$$

These metrics provide a balanced evaluation of the LLM's performance, considering both the correctness and completeness of the annotations. The distinction between unique requirements and individual annotations also allows for an overview of the models' performances as pure classifiers.

5.1.5 Handling Unmatched Annotations

During the matching process, if the number of annotations made by both models is not the same, the handling of annotations that cannot be matched must be defined.

- **Unmatched Reference Annotations:** Considered false negatives, indicating missed requirements by the contender.
- **Unmatched Contender Annotations:** Considered false positives, indicating incorrect or superfluous annotations.

5.2 Results

Due to differences during the detection of headlines and the resulting splitting of the policies into passages, the models have different amounts of passages that each attempted to annotate. The number of total passages analyzed by each model can be found in Table 5.1. The number stays the same across the unannotated and the annotated dataset for each model since no changes to the passage structure is made there. This provides some context for the rest of the results.

Model	Total Passages
llama3.1_8b-instruct-q4_0	1984
llama3.1_8b-instruct-fp16	2061
gemma2_9b-instruct-q4_0	1984
gemma2_27b-instruct-q4_0	2061
mistral-nemo_12b-instruct-2407-q4_0	1831
mixtral_instruct	2046
phi3_14b-medium-128k-instruct-q4_0	2061
qwen2_7b-instruct-fp16	1962

Table 5.1: Total Passages for Each Model (Reviewed and Annotated)

For the rest of the results, we will consider the values for the unreviewed annotations separately from the reviewed ones. Furthermore, all values given will be rounded to the 4th decimal place for brevity's sake. A table of all of the exact values can be found in Appendix 3.

5.2.1 Unreviewed Annotations

For the evaluation of the models' performance, a few metrics can be used to gain an overview of the data. These will be detailed individually in the following subsections.

5.2.1.1 Precision, Recall and F1-Score

Overall, the best performer in terms of precision, recall and F1-score was **Gemma 2 9B**. The model scored the highest for all three values, with a total precision score of 0.5984, a recall score of 0.6760 and an F1-score of 0.6348.

As Table 5.2 shows, the best performers in terms of precision apart from Gemma 2 9B are Gemma 2 27B, which takes second place in this metric with a score of 0.5561, and Mistral Nemo 12B, which comes in at third place with a score of 0.4998. Gemma 2 27B also comes in second in the recall category with a score of 0.6531, while Llama 3.1 8B with F16 quantization places third with a score of 0.6125. Lastly, regarding the F1-score, once again, Gemma 2 27B places second with a score of 0.6007 and Mistral Nemo 12B takes third place with an F1-score of 0.5296. This is also shown in Figure 5.1, where the best performer is highlighted in red.

Model	Precision	Recall	F1-Score
llama3.1:8b-instruct-q4_0	0.3268	0.5840	0.4191
llama3.1:8b-instruct-fp16	0.3449	0.6125	0.4413
gemma2:9b-instruct-q4_0	0.5984	0.6760	0.6348
gemma2:27b-instruct-q4_0	0.5561	0.6531	0.6007
mistral-nemo:12b-instruct-2407-q4_0	0.4998	0.5630	0.5296
mixtral:instruct	0.4096	0.5043	0.4521
phi3:14b-medium-128k-instruct-q4_0	0.3478	0.3911	0.3681
qwen2:7b-instruct-fp16	0.2871	0.5062	0.3664

Table 5.2: Performance Metrics

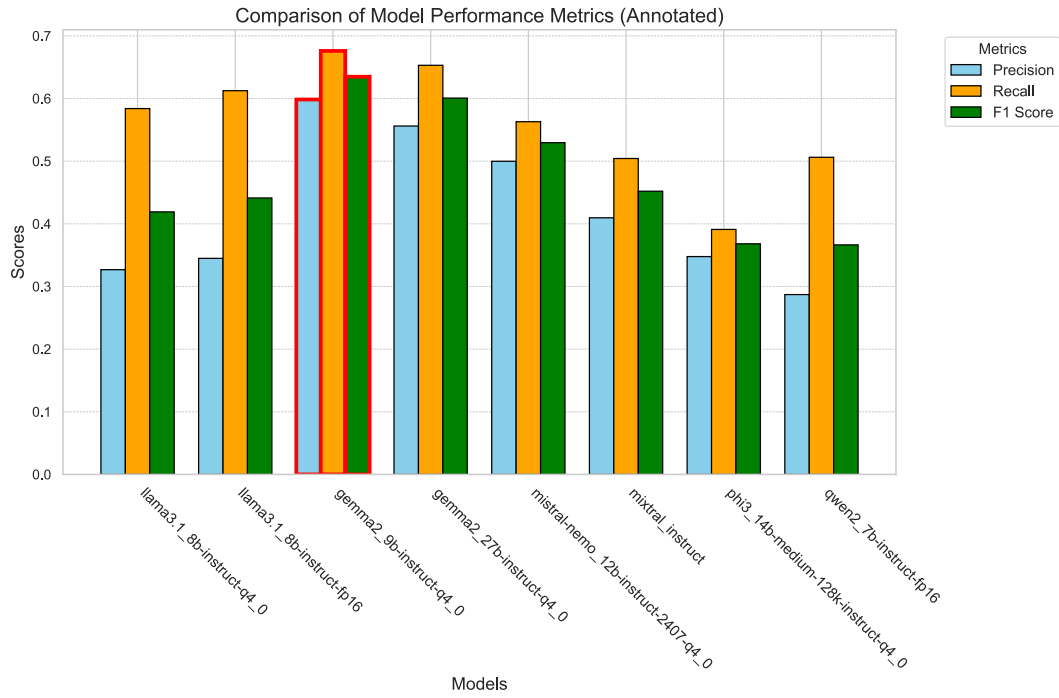


Figure 5.1: Performance Metrics

These metrics are calculated for individual annotations. They are also calculated separately as discussed in subsection 5.1.4.3 to evaluate the models' performances as classifiers. This can be found in Figure 5.2

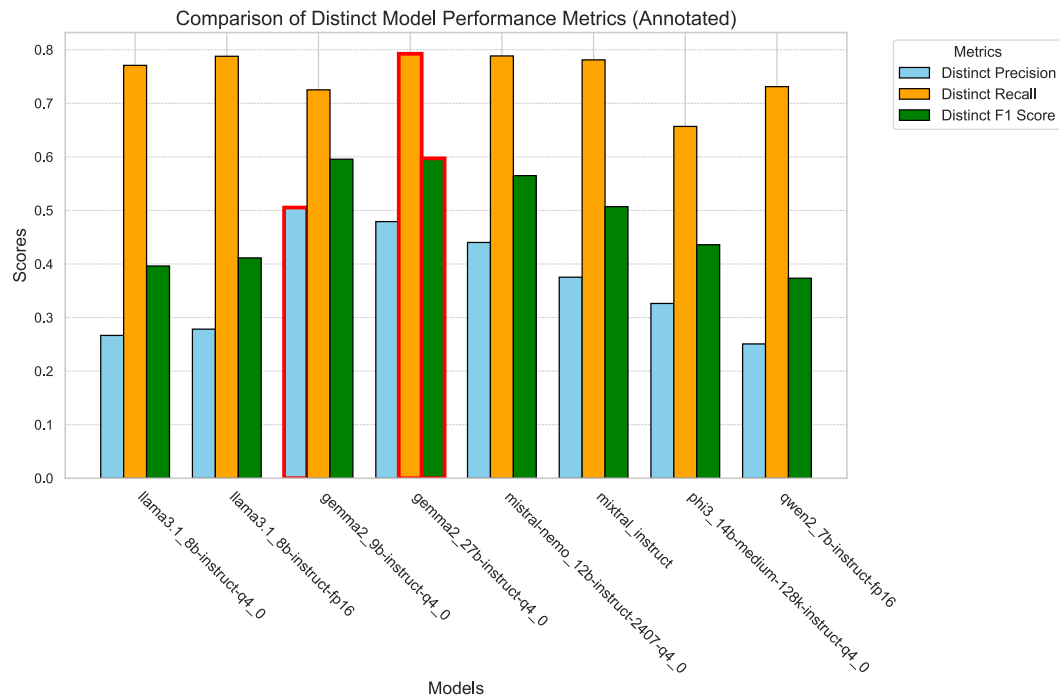


Figure 5.2: Distinct Performance Metrics

In terms of performance as a classifier, the previously highest scoring Gemma 2 9B is dethroned by the larger Google model, which performs minimally better in terms of F1-Score with a score of 0.5972 compared to the smaller model's 0.5956. Mistral's Nemo 12B comes in third with an F1-Score of 0.5650. As for recall, Gemma 2 27B also scores highest here, receiving a score of 0.7925, with Mistral Nemo 12B placing second (0.7884) and Llama 3.1 8B F16 coming in third (0.7878).

Between the two charts, the largest change in performance in each category is as follows:

- **Largest Difference in Precision:** Phi 3 14B displays a the largest change in precision between both types of precision, with it being a decrease of 0.0214. This metric stays fairly stable otherwise.
- **Largest Difference in Recall:** Mistral 8x7B exhibits the largest increase in recall from regular to distinct metrics, gaining 0.2768.
- **Largest Difference in F1-Score:** Phi 3 14B shows the highest change in F1 score (+0.0679) when shifting from regular to distinct metrics.

5.2.1.2 F1 Score Sorted by Model Size

Another interesting potential correlation could be between the models' sizes. Figure 5.3 shows their F1-Scores, with the models being sorted by parameter count. It is vital to note here that for Mistral 8x7B, which is shown as having 7 billion parameters, it is debatable if it should be classified as a 7B model or a 56B model, seeing as its Mixture of Experts architecture "routes" the prompt to an appropriate 7B model, rather than fully utilizing all 56 billion parameters.

However, while it does *potentially* have access to 56 billion parameters, each prompt is only processed by a 7B model, which is why it was ranked as a 7B model in this plot. This plot shows no clearly visible correlation between model size and performance, with Google's smaller model beating the larger model, and other larger models not performing significantly better than the smaller ones.

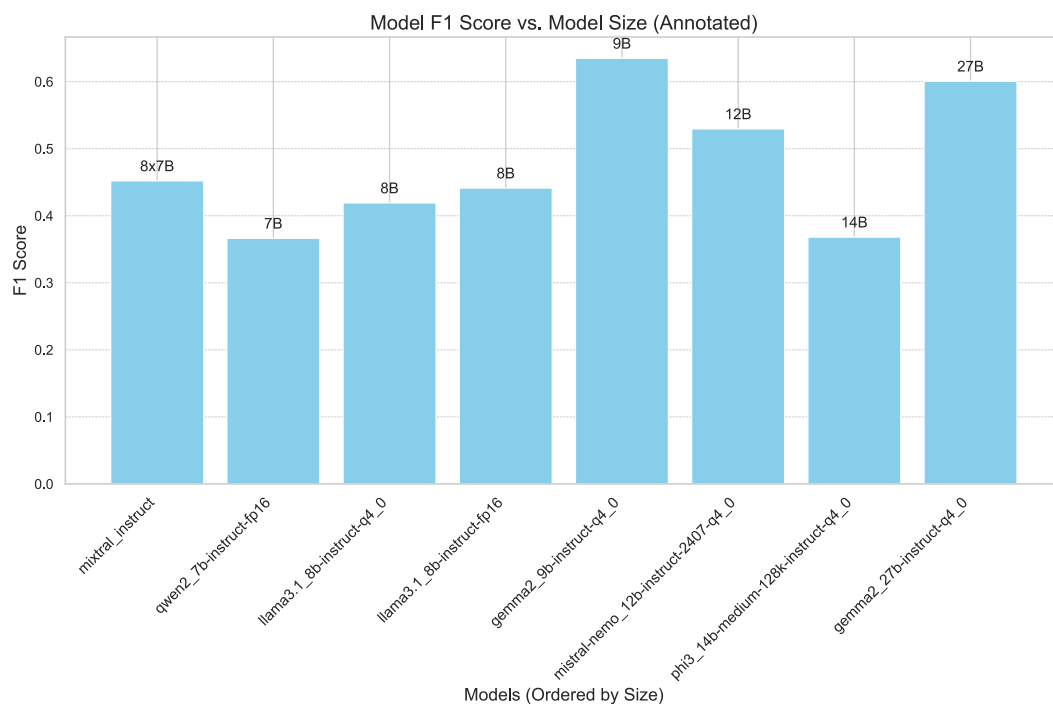


Figure 5.3: Model F1-Scores, Sorted by Model Size

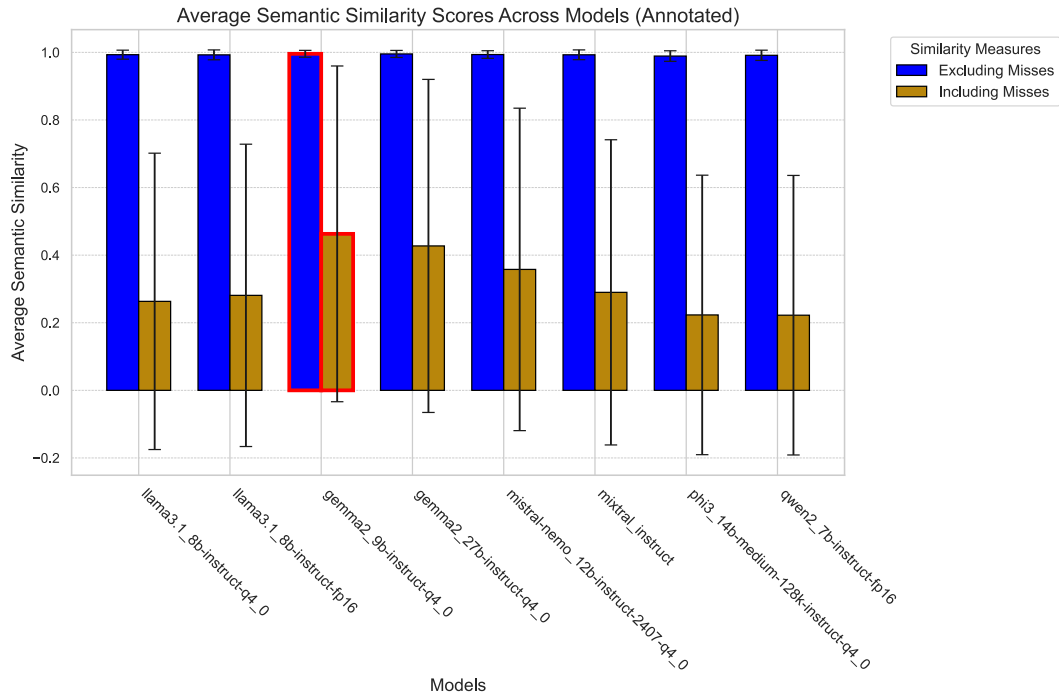
5.2.1.3 Average Similarity Scores

During the course of the evaluation, we calculated the average similarity scores achieved by every model, displayed in Table 5.3. The first column contains the Average Semantic Similarity score (abbreviated as "ASS"), the second column contains the Average Word-Level Similarity score (AWLS), the third represents the Average Semantic Similarity including Misses (ASSM), i.e. the average semantic similarity score, but with False Positives and False Negatives scored as a similarity score of 0, and the fourth column shows the Average Word-Level Similarity Score including Misses (AWLSSM).

Model	ASS	AWLS	ASSM	AWLSSM
llama3.1:8b-instruct-q4_0	0.9931	0.8103	0.2633	0.2148
llama3.1:8b-instruct-fp16	0.9926	0.8073	0.2810	0.2285
gemma2:9b-instruct-q4_0	0.9957	0.8724	0.4630	0.4057
gemma2:27b-instruct-q4_0	0.9953	0.8486	0.4273	0.3643
mistral-nemo:12b-instruct-2407-q4_0	0.9936	0.8078	0.3578	0.2909
mixtral:instruct	0.9928	0.7741	0.2899	0.2261
phi3:14b-medium-128k-instruct-q4_0	0.9891	0.7025	0.2231	0.1585
qwen2:7b-instruct-fp16	0.9913	0.7581	0.2223	0.1700

Table 5.3: Annotated Data: Average Similarity Scores

Here (Table 5.3), Gemma 2 9B tops all categories again, with its larger sibling Gemma 2 27B following right after, scoring second in all categories, and Mistral Nemo 12B coming in at third place in all scores except for average word-level similarity, where Llama 3.1 8B with Q4 quantization places third.

**Figure 5.4:** Average Semantic Similarity Scores (with Standard Deviation)

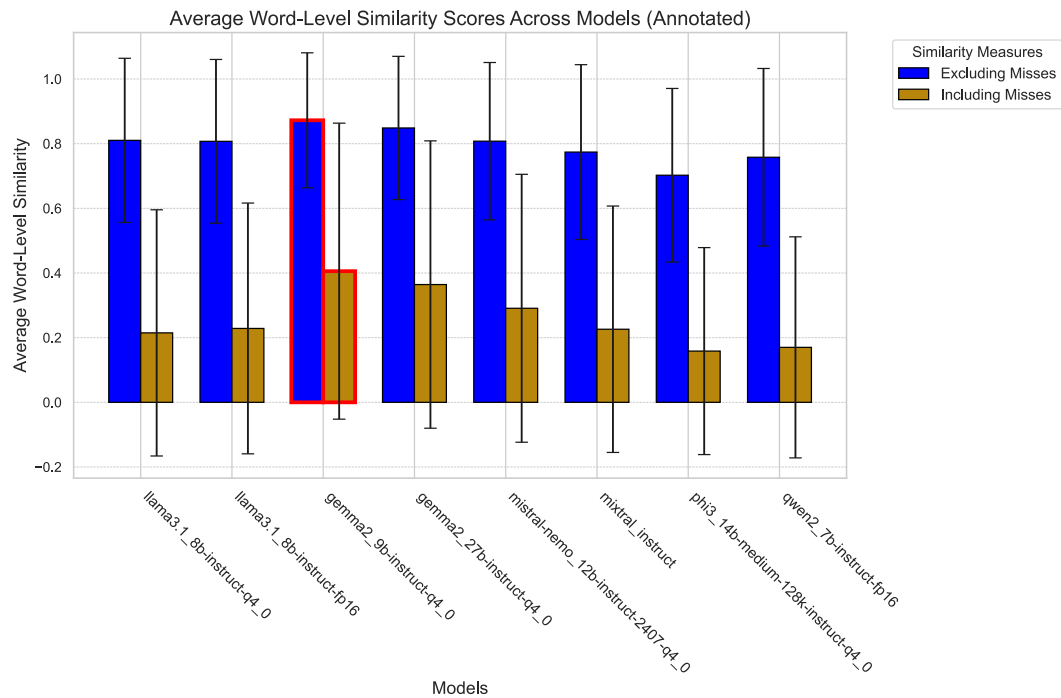
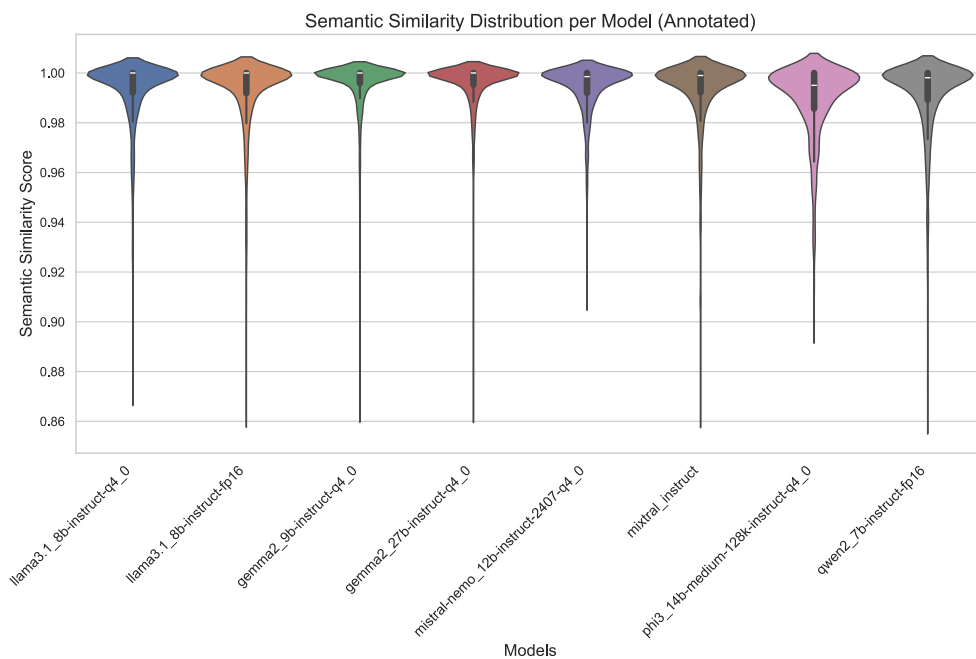
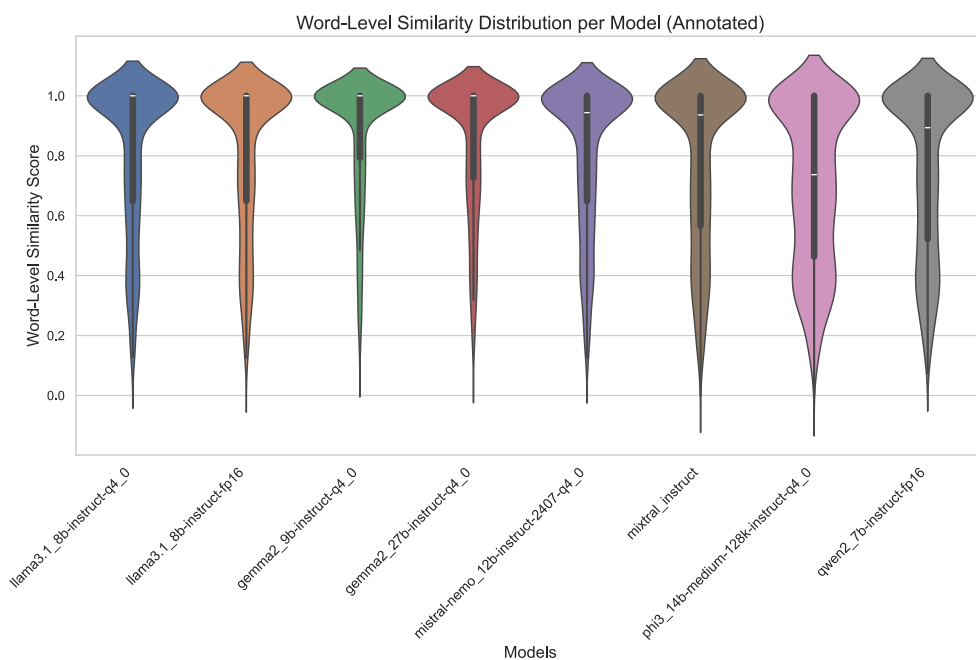


Figure 5.5: Average Word-Level Similarity Scores (with Standard Deviation)

Figures 5.4 and 5.5 show these values as bar charts with the corresponding standard deviations displayed as error bars for each metric. Again, the best performer is highlighted in red, which, in this case, is Gemma 2 9B.

In the same context, it can be helpful to also visualize the distribution of the average similarity scores. Since most models receive very high average semantic similarity scores, it can be hard to differentiate between the models' performances using only the average. Figure 5.6 shows a violin plot of the semantic similarity scores of the models, revealing that Phi 3 14B is noticeably elongated compared to the other models displayed. This trend is reinforced when looking at the word-level similarity distributions, shown in Figure 5.7. Here, it is apparent that the two Gemma models make annotations for values that are the most similar in wording to the reference model, with the two tested Llama models and Mistral Nemo following after, and Mistral, Phi 3 and Qwen2 bringing up the rear.

**Figure 5.6: Semantic Similarity Distribution****Figure 5.7: Word-Level Similarity Distribution**

5.2.1.4 Average Correct Annotations Ratio

This metric, defined in Equation 5.5, serves to provide an overview of which models are best at identifying all of the correct requirements in a passage, regardless of how often they occur and not taking the quality of the annotations into account. This is displayed in Figure 5.8, which reveals that, once again, Gemma 2 9B scores highest with a score of 0.6400. Gemma 2 27B is the runner-up with an average score of 0.6212 and Qwen2 7B with F16 quantization places third with an average ratio of 0.6105. Models that score high on this score demonstrate a better ability to act as a classifier, for example in a task where a list of requirements is given alongside a passage and the LLM is tasked with identifying all requirements pertinent to the passage.

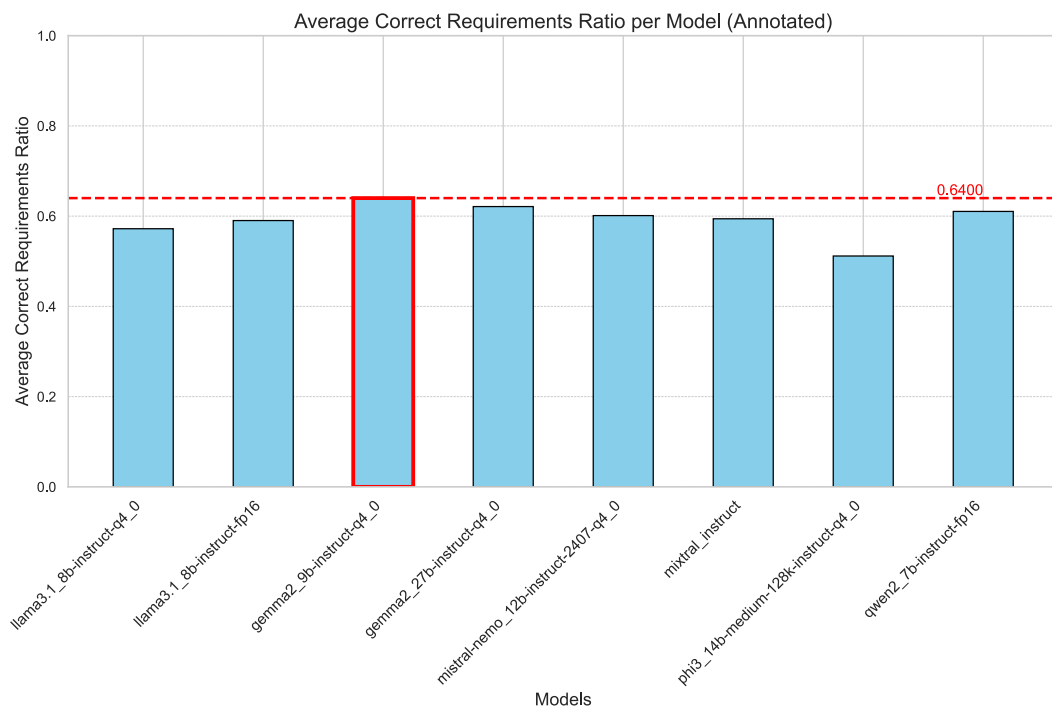
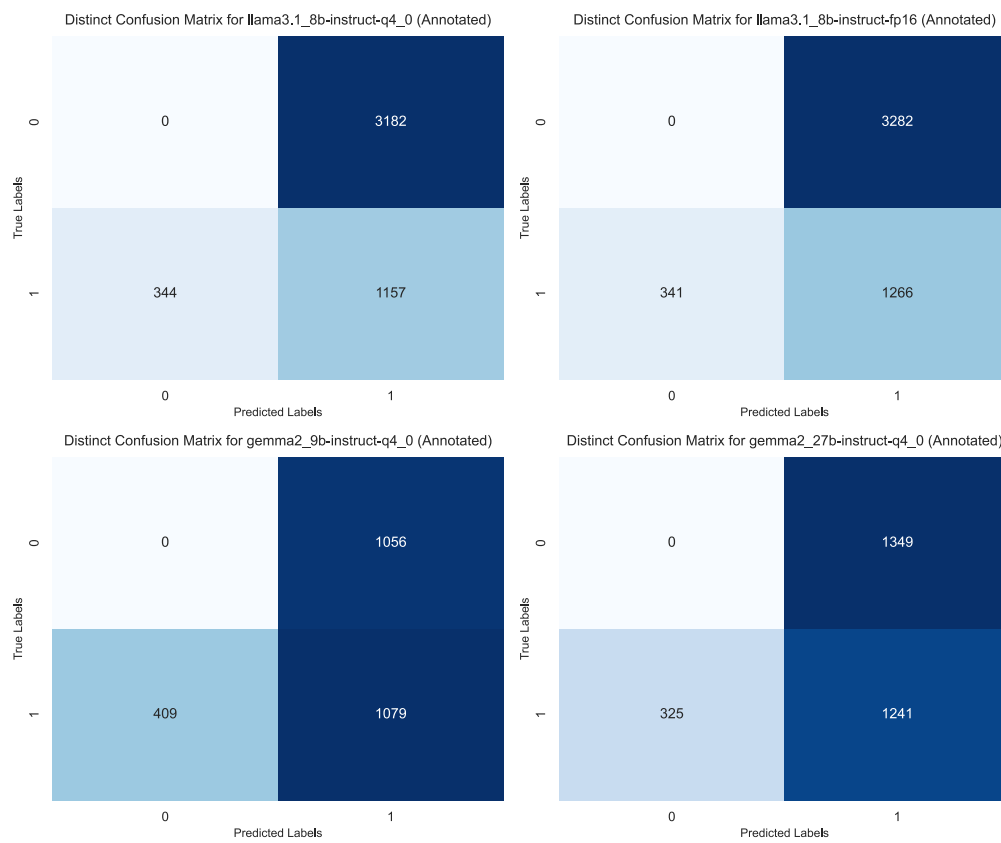


Figure 5.8: Average Correct Requirements Ratio per Model

5.2.1.5 Confusion Matrices

In order to gain an overview of the absolute numbers of true positive, false negative, and false positive numbers that make up the F1-Score, confusion matrices can be considered. These show the proportions of true positives, false positives and false negatives, as shown in Figures 5.9 and 5.10. In theory, this chart would also contain true negatives, but since there is no labeling of *wrong* requirements, this is omitted. These charts can give a sense of whether models tend to over-annotate or over-annotate. For the purpose of this plot, we use the distinct requirements annotated, interpreting the annotations as a classification task.

**Figure 5.9:** Distinct Confusion Matrices: Part 1

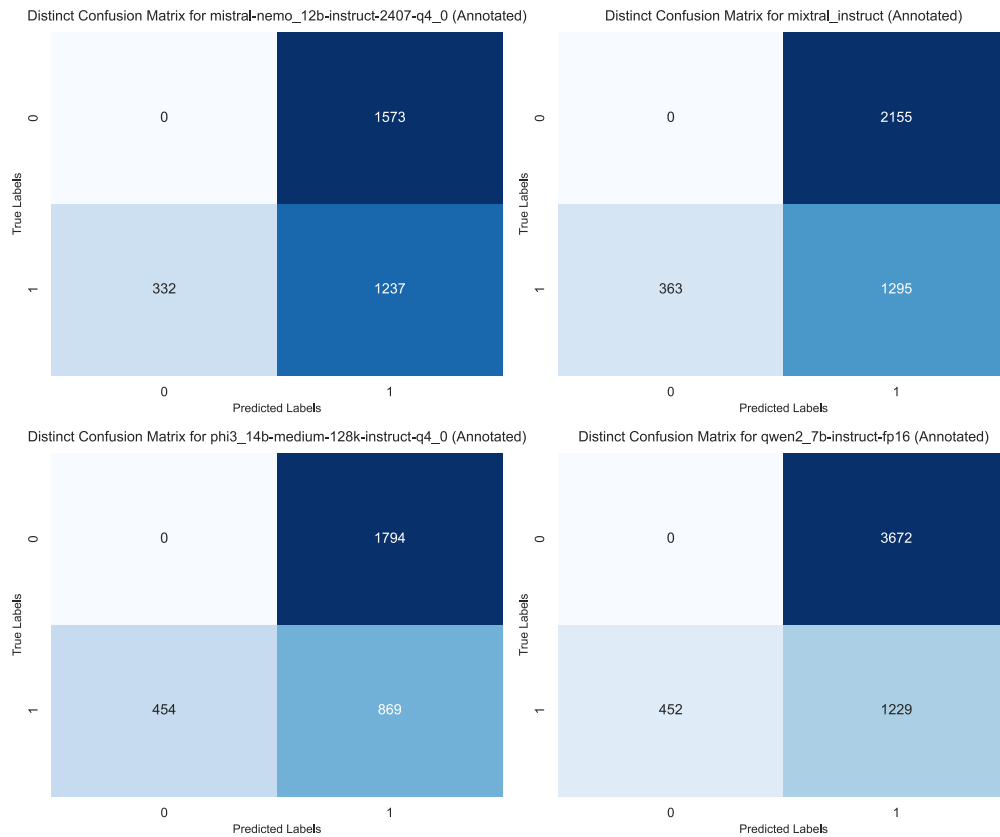


Figure 5.10: Distinct Confusion Matrices: Part 2

An evaluation of these confusion matrices shows, for example, that the highest scoring model in terms of distinct F1-Score, Gemma 2 27B, gets a large proportion of annotations correct, but also over-annotates quite a bit. In contrast, Gemma 2 9B over-annotates less than its sibling model, but misses more annotations proportionally than the larger model. On the other end of the spectrum, Qwen2 7B, for instance, tends to make a lot of annotations that the reference model does not make.

In a slightly different look, it can also be helpful to look at these matrices from a different perspective that displays the relative proportions of the components differently. This can be seen in Figure 5.11. Here, the components of the matrices are stacked on top of each other, enabling better comparability across models.

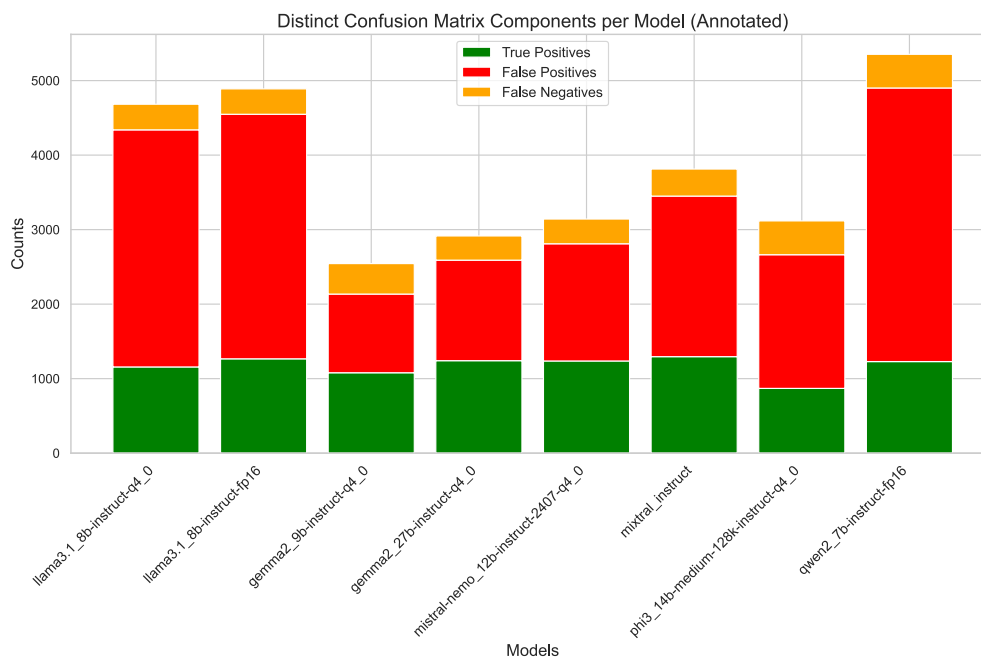


Figure 5.11: Stacked Confusion Matrix Components

5.2.1.6 Combined Visualizations

Finally, the metrics used to assess the models can also be combined into a select number of plots that show multiple metrics together. This provides an overview of general performance, and can allow for the recognition of patterns and potential correlations. First, the main metrics can be combined into a heat map chart, shown in Figure 5.12. This figure shows, for instance, that all models consistently score higher on recall than precision and F1.

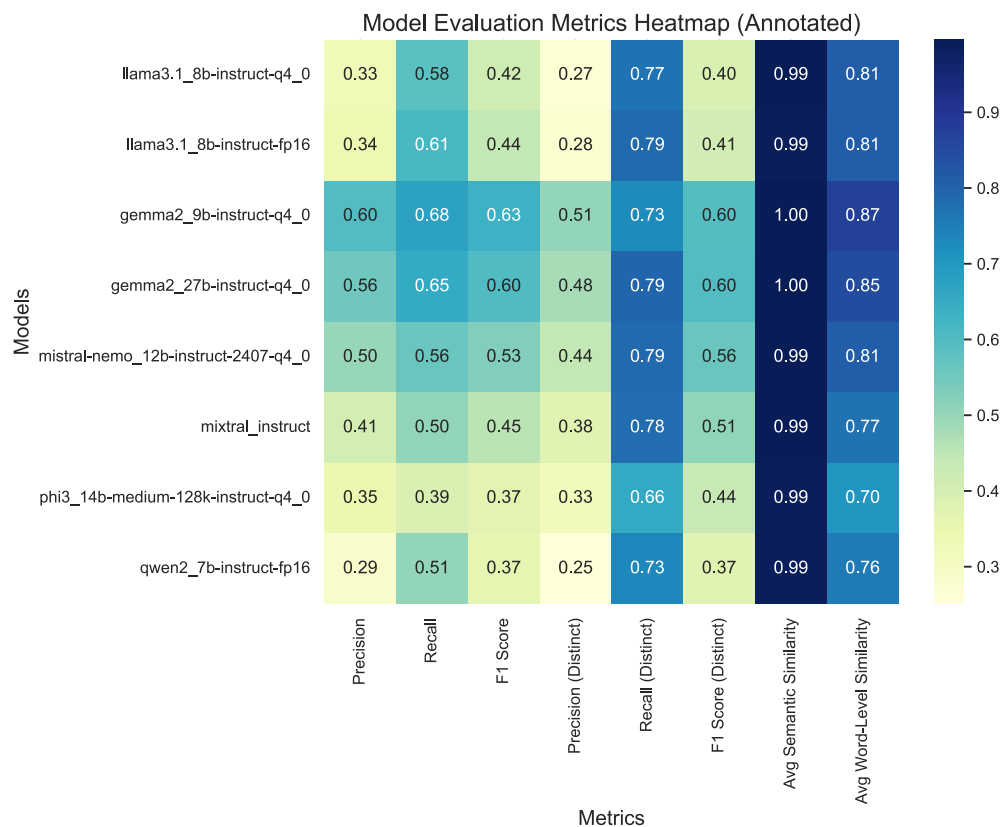


Figure 5.12: Model Metric Heat Map

A different type of graph that also serves to provide a better sense of overall performance is a radar chart, as seen in Figure 5.13. Here, general performance can be assessed by the total area a model covers on the chart: the more, the higher the scores. It shows that Gemma 2 9B scores the highest in almost all metrics, with Gemma 2 27B coming second in most cases and Mistral Nemo and Llama 3.1 8B with F16 quantization also being among the top scorers in a few categories.

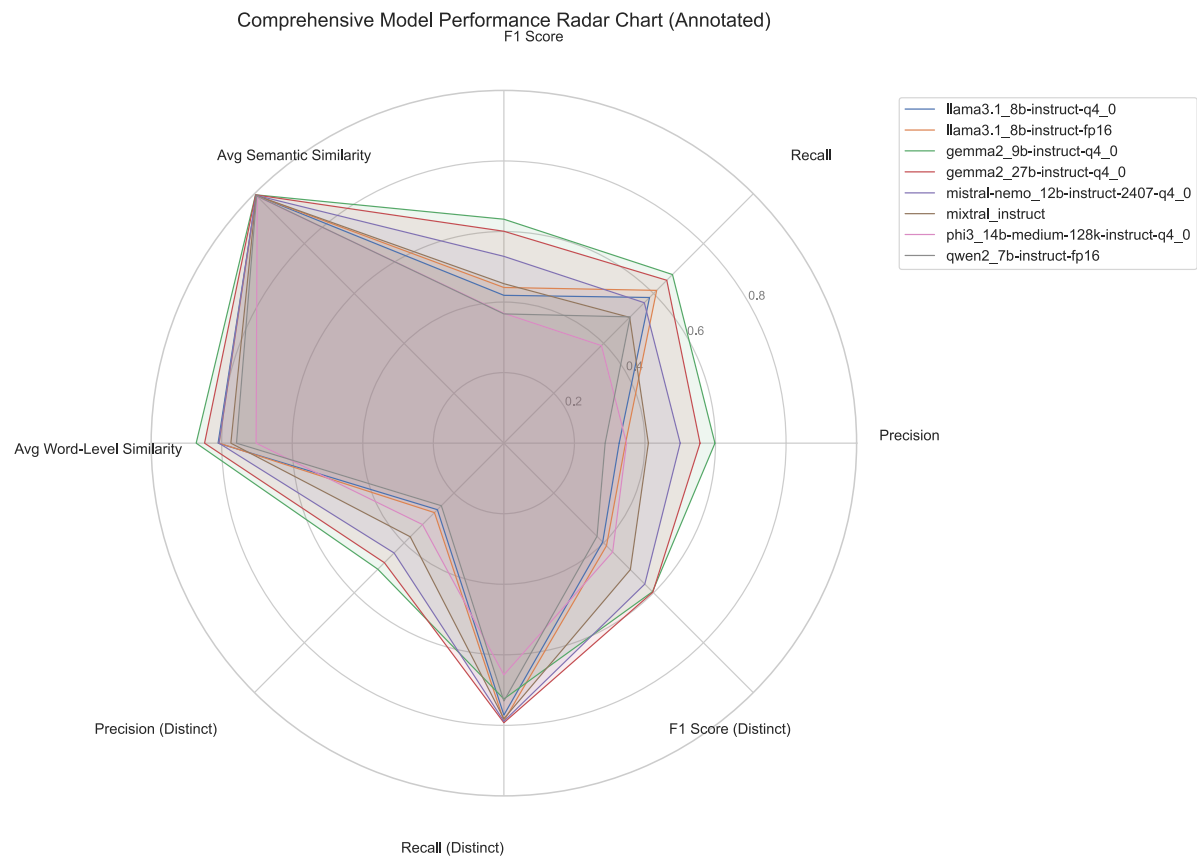


Figure 5.13: Model Metric Radar Chart

5.2.2 Reviewed Annotations

The pipeline used does not only produce regular annotations, but also reviewed annotations in an effort to improve the quality of the annotations. Therefore, it is necessary to consider the metrics for the reviewed annotations as well. For additional plots that were discussed for the unreviewed annotation and are not mentioned here, reference Appendix 1, as this section will omit some plots to avoid excessive repetition.

5.2.2.1 Precision, Recall and F1-Score

As evidenced by Table 5.4, Google’s Gemma 2 9B scores highest in precision and F1-Score, while Meta’s Llama 3.1 8B scores highest in recall. The runners-up in precision are Gemma 2 27B and Llama 3.1 8B F16, Gemma 2 9B and Gemma 2 27B in terms of recall, and Gemma 2 27B as well as Llama 3.1 8B F16 for F1-Score.

Model	Precision	Recall	F1-Score
llama3.1:8b-instruct-q4_0	0.4218	0.5187	0.4652
llama3.1:8b-instruct-fp16	0.4879	0.4654	0.4764
gemma2:9b-instruct-q4_0	0.5774	0.5706	0.5740
gemma2:27b-instruct-q4_0	0.5333	0.5158	0.5244
mistral-nemo:12b-instruct-2407-q4_0	0.4401	0.4891	0.4633
mixtral:instruct	0.3871	0.4385	0.4112
phi3:14b-medium-128k-instruct-q4_0	0.3492	0.3886	0.3678
qwen2:7b-instruct-fp16	0.1785	0.4101	0.2487

Table 5.4: Performance Metrics

5.2.2.2 F1-Score Sorted by Model Size

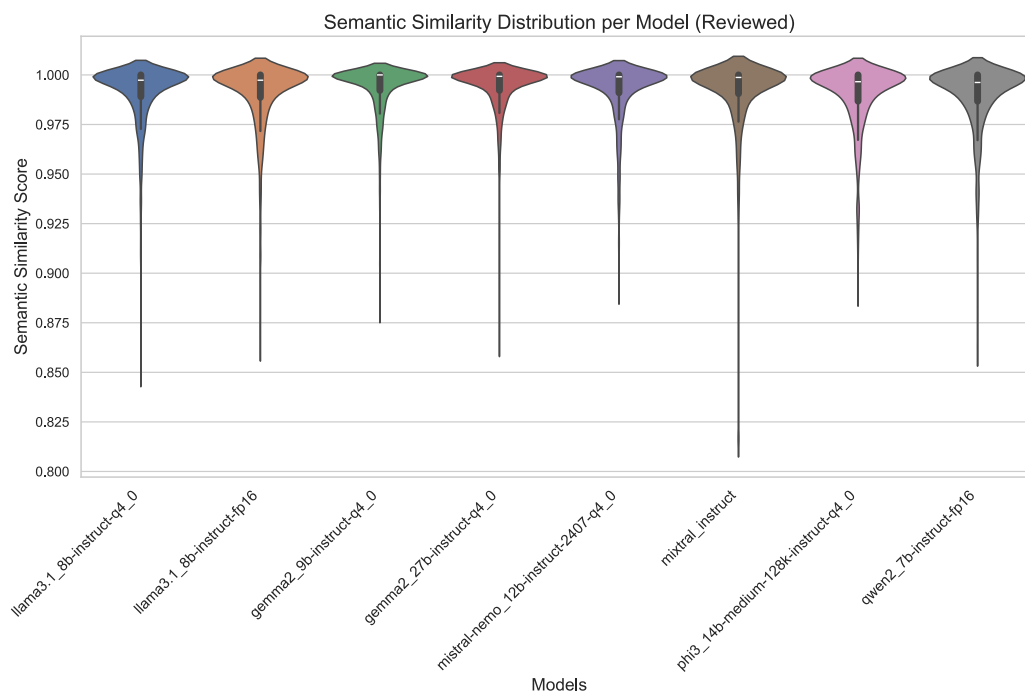
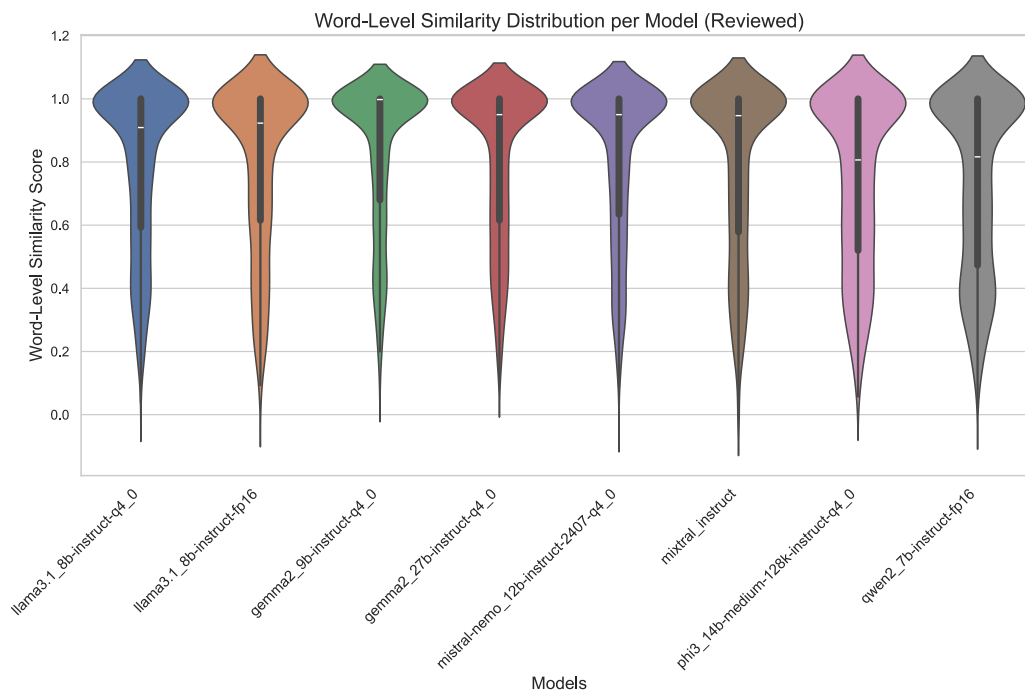
This chart can be found in Appendix 1, Figure 3 and does not indicate any correlation of model performance and model size.

5.2.2.3 Average Similarity Scores

For a table of the exact average similarity scores as discussed for the unreviewed annotations, please reference Appendix 3, Table 4. It shows that Gemma 2 9B scores highest in all similarity score categories, with its larger sibling Gemma 2 27B following suit and the two Llama models as well as Mistral Nemo forming the middle field.

The charts for these metrics, both for the classification of distinct requirement categories as well as the regular scores, can be found in Appendix 1, Figures 1 and 2.

As for the distribution of semantic similarity scores, which can be viewed in Figure 5.14, Mixtral 8x7B has a noticeably longer tail than the rest of the models. Word-level similarity distributions are fairly balanced for all of the models, as seen in Figure 5.15. Further plots can also be found in Appendix 1, Figures 4 and 5.

**Figure 5.14: Semantic Similarity Distribution****Figure 5.15: Word-Level Similarity Distribution**

5.2.2.4 Average Correct Annotations Ratio

In this metric, Gemma 2 9B scores highest again, with a score of 0.6415, with Llama 3.1 8B F16 coming in second with a score of 0.6147 and Llama 3.1 8B taking third place with a score of 0.6389. For an overview of these scores, reference Appendix 1, Figure 6.

5.2.2.5 Confusion Matrices

For the exact confusion matrices for the reviewed annotations, reference Appendix 1, Figures 7 and 8. Taking a look at the stacked confusion matrices for the reviewed annotations, shown in Figure 5.16, Qwen2 7B appears as a clear outlier, having made significantly more annotations than the other models, with more than 6000 requirements identified by it, compared to Llama 3.1 8B F16's 1451 distinct requirements annotated.

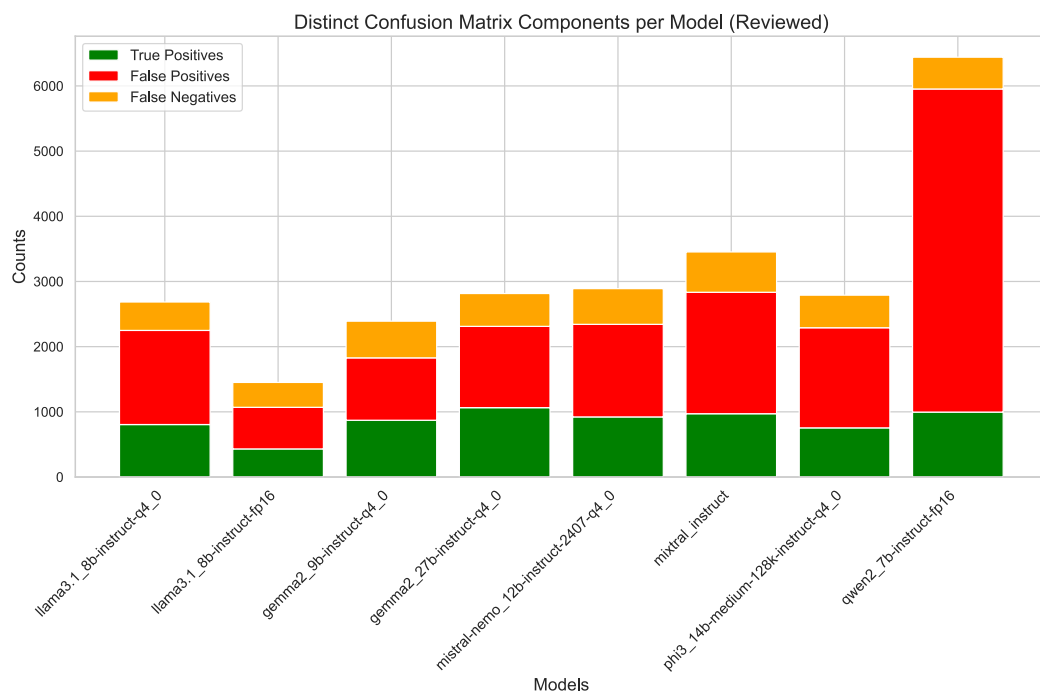


Figure 5.16: Stacked Confusion Matrix Components

5.2.2.6 Combined Visualizations

For a heat map containing the combined performance metrics of the reviewed annotations, refer to Appendix 1, Figure 5.12.

The radar chart of the different metrics, seen in Figure 5.17, clearly show that Qwen2 7B scores extremely low on most metrics, while Gemma 2 9B takes first place in most measures. Gemma 2 27B consistently places top 3 in almost all metrics.

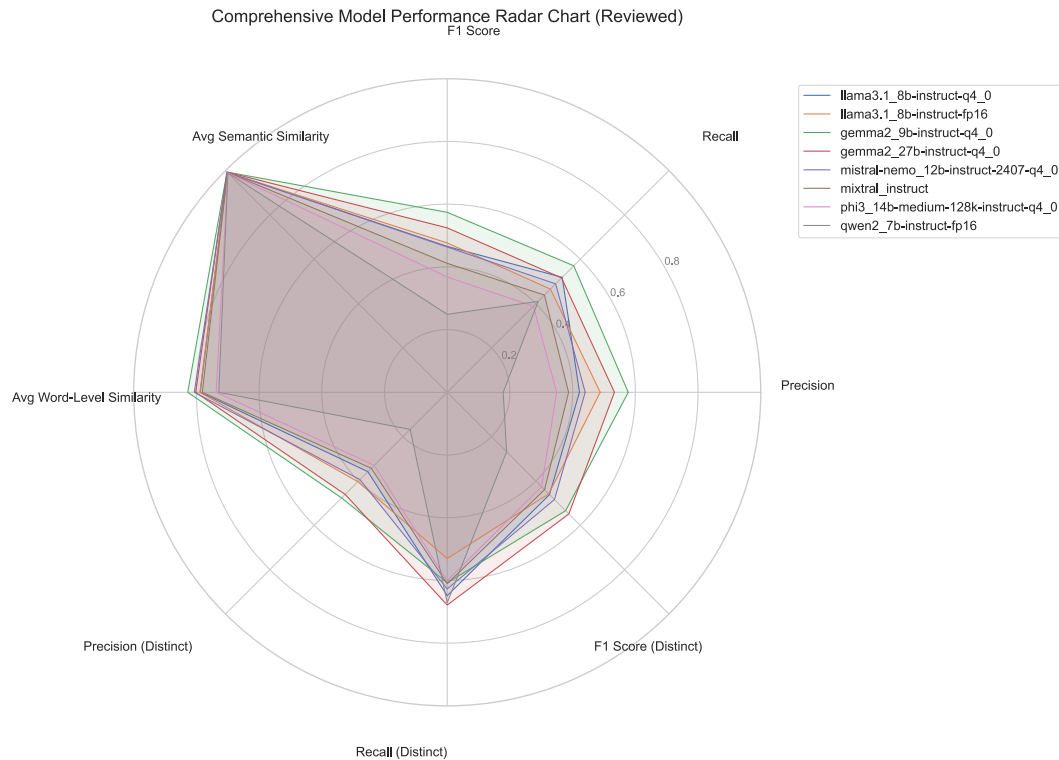


Figure 5.17: Model Metric Radar Chart (Reviewed Annotations)

5.3 Discussion

The evaluation of the LLMs has highlighted significant trends and disparities in performance. Notably, **Gemma 2 9B** consistently emerged as the top performer across multiple metrics, including precision, recall, F1-score, and average similarity scores. Its ability to accurately identify and closely replicate relevant requirements on a word-for-word level within policy documents suggests that its architecture is well suited for this type of annotation task. This performance may be attributed to targeted training on policy-related texts or specific emphasis on requirement extraction tasks, enhancing its domain-specific proficiency. The notion that its architecture or training data could be more well picked or suited for legal document annotation is reinforced by the fact that its larger sibling also scores well in most metrics, often placing second.

However, the larger Gemma 2 27B did not outperform its smaller counterpart for the most part, indicating that simply increasing model size does not necessarily translate to better performance in specialized tasks like requirement extraction. This observation is further reinforced by the underperformance of Phi 3 14B, which, despite having the second highest parameter count among the tested models, exhibited lower precision and F1-scores compared to smaller models. The higher variance in its annotation quality, as evidenced by elongated tails in similarity distribution plots, suggests that increased complexity may introduce challenges in fine-tuning and lead to overfitting or underfitting if not properly managed.

A particularly striking finding pertains to Qwen2 7B, which not only performed poorly

across most metrics but also generated an abnormally high number of annotations compared to other models. While most models saw a significant reduction in the total number of annotations after the review process (by around half for Llama 3.1 8B Q4 and from nearly 5,000 down to approximately 1,500 for Llama 3.1 F16), Qwen2 7B's annotations *increased* by about 1,000 post-review. This suggests that Qwen2 7B lacks effective mechanisms for discriminating between relevant and irrelevant content, leading to a high false positive rate that was not mitigated by the review process. The model seems eager to annotate as many sections as possible, as evidenced by the extremely high amount of false positive annotations made by it, especially after review. This might suggest a bias within the training data, or might even be a tendency introduced during the instruct-finetuning phase the model underwent, during which it might have been trained to follow the given instructions without critically questioning its own output, leading to it wanting to generate annotations at any cost.

These trends underscore that factors such as training data relevance, model architecture, and fine-tuning efficacy play more critical roles in model performance than sheer size. The observation that the total number of annotations generally decreases after reviewing for most models indicates that the review process effectively filters out incorrect annotations, improving overall precision. However, the inverse result for Qwen2 7B emphasizes the need for models to have robust initial annotation quality to benefit from such a review process. Additionally, the F1-Scores for the models also decreases after reviewing, signifying that correct annotations may be filtered out during the process. This effect, however, is likely attributed to the reviewing process itself, and specifically the system prompt used, which can be easily adjusted during future research.

One benefit of using open-source LLMs has proven to be their customizability and adaptability. The large variety of available open-source LLMs, finetuned variations for different tasks and knowledge domains, and multitude of available model sizes presents the opportunity for efficient policy processing given some more developments either in model architecture or pipeline efficacy. New models such as the newly released (at the time of writing) Llama 3.2, which is available both in a 3B and a 1B size, could open up interesting possibilities regarding the analysis of privacy policies on lower-end hardware, or the large scale parallel analysis of a significantly bigger set of policies, should they prove to be effective for this application.

Overall, the evaluation reveals that while Gemma 2 9B achieves the best performance, outperforming larger models and highlighting the importance of targeted training and architectural balance, it still leaves a lot to be desired in terms of reliability, with its F1-Score before reviewing being only around 0.6. The poor performance of models like Qwen2 7B demonstrates that the performance of LLMs for privacy policy annotation is highly dependent on the training of the models. This is supported by the fact that models from the same families, i.e. the Gemma models, Llama models and Mistral AI-produced models typically showed somewhat similar performance, indicating that their likely similar training data and process is highly influential towards their suitability. These findings suggest that future efforts should focus on finding the model architectures best suited to the specific complexities of policy language and requirement extraction tasks, rather than solely increasing model size, and potentially finetuning a model for the specific annotation task, as this could prove to be highly beneficial to its performance.

5.3.1 Limitations and Potential Sources of Error

One limitation that may have affected the results is the variation in passage segmentation due to differences in headline detection among the models. As each model processed a different number of passages (Table 5.1), this inconsistency could introduce variability in the number and content of annotations. Models analyzing more passages had more opportunities to identify requirements, potentially affecting recall and precision metrics.

Another factor to consider are the inherent ambiguity in policy language and the subjective nature of requirement interpretation can lead to inconsistencies in annotations. Models may struggle with nuanced language, legal jargon, or complex sentence structures commonly found in policy documents. This challenge is exacerbated when models lack sufficient domain-specific training data, something likely a factor in the poor performance of the Alibaba- and Microsoft-trained models.

Moreover, the evaluation metrics rely on reference annotations that may themselves be imperfect or biased. Even though GPT-4o is one of the most sophisticated models currently available, it is also subject to the same randomness as the open-source models evaluated here. Any errors or omissions in the reference set could unfairly penalize models, particularly if they correctly identify requirements not captured by the reference. In the future, datasets that are human-validated or generated by even more advanced models that might emerge, such as OpenAI's new o1 and o1-mini models that have been recently released at the time of writing, could be considered to eliminate this error factor. The evaluation could also benefit from the use of a larger dataset of policies, potentially averaging out error over even larger sample sizes.

6 Conclusion

In this thesis, an extensive evaluation of various large language models was conducted with the goal of enhancing the understanding of their capabilities in policy analysis and requirement extraction tasks. The models examined ranged from smaller architectures like Qwen2 7B to larger ones like Phi 3 14B and Gemma 2 27B, encompassing a diverse spectrum of parameter counts and training backgrounds.

The methodology involved processing policy documents to extract requirements, utilizing both unreviewed and reviewed annotations to assess the models' performance comprehensively. Metrics such as precision, recall, F1-score, and average similarity scores were employed to provide a detailed and comparable evaluation of each model's effectiveness. The reference annotations were generated using GPT-4o, serving as a consistent benchmark across all evaluations.

The results demonstrated that **Gemma 2 9B** consistently outperformed other models across most metrics, albeit typically not by a large margin compared to the second- and third-placed models. Its architecture and training appear to offer an optimal blend of efficiency and capacity to capture complex language patterns inherent in policy documents. This suggests that targeted training and architectural design significantly enhance a model's proficiency in specialized tasks like requirement extraction.

Interestingly, increasing the model size did not necessarily correlate with improved performance, though the sample range used was not as extensive as it should be for a definitive evaluation of this correlation and therefore may not be as significant. Despite having more parameters, models like Gemma 2 27B and Phi 3 14B did not surpass the 9B variant. Conversely, models like **Qwen2 7B** underperformed significantly. The anomaly of its annotations increasing after the review process, in contrast to the general trend of reductions seen in other models, highlights deficiencies in its ability to discriminate between relevant and irrelevant content. This emphasizes the critical role of effective training data and finetuning procedures tailored to the area of application.

The review process itself proved effective for most models, reducing the total number of annotations and enhancing overall precision. However, the decrease in F1-scores post-review suggests that some correct annotations may have been inadvertently filtered out. This points to potential limitations in the review methodology or the necessity for more sophisticated system prompts to better preserve valid annotations.

Several limitations were identified in this thesis. Variations in passage segmentation due to differences in headline detection among the models introduced inconsistencies that could affect the comparability of results. Additionally, relying on reference annotations generated by GPT-4o, while leveraging one of the most advanced models available, carries the risk of propagating any inherent biases or errors present in its outputs. Future research could focus

on fine-tuning models specifically for policy analysis and requirement extraction to further enhance performance. Exploring alternative architectures optimized for legal and policy language understanding may prove more beneficial than increasing model size alone. Incorporating human-validated annotations and expanding the dataset to include a broader range of policies would also strengthen the robustness of evaluations. Larger sample sizes of models or policies were not realistic within the scope of this project due to run time of the pipeline (around 35 hours for the full analysis of all 50 policies and 8 models), which is another limiting factor that, as discussed in Chapter 4, could be alleviated by using better hardware or by implementing performance enhancements in the pipeline itself.

List of Tables

4.1	LLMs used in the Project	19
5.1	Total Passages for Each Model (Reviewed and Annotated)	35
5.2	Performance Metrics	36
5.3	Annotated Data: Average Similarity Scores	39
5.4	Performance Metrics	48
1	Metrics from Annotated Dataset	69
2	Similarity Scores from Annotated Dataset	69
3	Metrics from Reviewed Dataset	70
4	Similarity Scores from Reviewed Dataset	70

List of Figures

3.1	Project Pipeline	13
5.1	Performance Metrics	36
5.2	Distinct Performance Metrics	37
5.3	Model F1-Scores, Sorted by Model Size	38
5.4	Average Semantic Similarity Scores (with Standard Deviation)	39
5.5	Average Word-Level Similarity Scores (with Standard Deviation)	40
5.6	Semantic Similarity Distribution	41
5.7	Word-Level Similarity Distribution	41
5.8	Average Correct Requirements Ratio per Model	42
5.9	Distinct Confusion Matrices: Part 1	43
5.10	Distinct Confusion Matrices: Part 2	44
5.11	Stacked Confusion Matrix Components	45
5.12	Model Metric Heat Map	46
5.13	Model Metric Radar Chart	47
5.14	Semantic Similarity Distribution	49
5.15	Word-Level Similarity Distribution	49
5.16	Stacked Confusion Matrix Components	50
5.17	Model Metric Radar Chart (Reviewed Annotations)	51
1	Performance Metrics (Reviewed Annotations)	71
2	Distinct Performance Metrics (Reviewed Annotations)	72
3	Model F1-Scores, Sorted by Model Size (Reviewed Annotations)	72
4	Average Semantic Similarity Scores (with Standard Deviation) (Reviewed Annotations)	73
5	Average Word-Level Similarity Scores (with Standard Deviation) (Reviewed Annotations)	73
6	Average Correct Requirements Ratio per Model (Reviewed Annotations)	74
7	Distinct Confusion Matrices: Part 1 (Reviewed Annotations)	75
8	Distinct Confusion Matrices: Part 2 (Reviewed Annotations)	76
9	Model Metric Heat Map (Reviewed Annotations)	77

Bibliography

- [1] R. Koch. (2024) What is the LGPD? Brazil’s version of the GDPR. [Online]. Available: <https://gdpr.eu/gdpr-vs-lgpd/>
- [2] CMS. (2024) GDPR Enforcement Tracker. [Online]. Available: <https://www.enforcementtracker.com/ETid-1844>
- [3] B. Fabian, T. Ermakova, and T. Lentz, “Large-scale readability analysis of privacy policies,” in *Proceedings of the international conference on web intelligence*, 2017, pp. 18–25.
- [4] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O’Reilly Media, Inc., 2009.
- [5] K. Chowdhary and K. Chowdhary, “Natural language processing,” *Fundamentals of artificial intelligence*, pp. 603–649, 2020.
- [6] J. Weizenbaum, “Eliza—a computer program for the study of natural language communication between man and machine,” *Communications of the ACM*, vol. 9, no. 1, pp. 36–45, 1966.
- [7] X. Liu and D. Wu, “Pie: programming in eliza,” in *Proceedings of the 29th ACM/IEEE international conference on Automated software engineering*, 2014, pp. 695–700.
- [8] M.-F. Moens, C. Uyttendaele, and J. Dumortier, “Abstracting of legal cases: The SALOMON experience,” in *Proceedings of the 6th international conference on Artificial Intelligence and Law*, 1997, pp. 114–122.
- [9] T. Mullen and N. Collier, “Sentiment analysis using support vector machines with diverse information sources,” in *Proceedings of the 2004 conference on empirical methods in natural language processing*, 2004, pp. 412–418.
- [10] S. Tong and D. Koller, “Support vector machine active learning with applications to text classification,” *Journal of machine learning research*, vol. 2, no. Nov, pp. 45–66, 2001.
- [11] T. Joachims *et al.*, “Transductive inference for text classification using support vector machines,” in *Icml*, vol. 99, 1999, pp. 200–209.
- [12] E. Osuna, R. Freund, and F. Girosit, “Training support vector machines: an application to face detection,” in *Proceedings of IEEE computer society conference on computer vision and pattern recognition*. IEEE, 1997, pp. 130–136.
- [13] E. Y. Boateng, D. A. Abaye *et al.*, “A review of the logistic regression model with emphasis on medical research,” *Journal of data analysis and information processing*, vol. 7, no. 04, p. 190, 2019.
- [14] J. Lee, “An insight on the use of multiple logistic regression analysis to estimate association between risk factor and disease occurrence,” *International journal of epidemiology*, vol. 15, no. 1, pp. 22–29, 1986.
- [15] F. Rosenblatt, “Principles of neurodynamics. perceptrons and the theory of brain mechanisms,”

Cornell Aeronautical Lab Inc Buffalo NY, Tech. Rep., 1961.

- [16] P. J. Werbos, "Applications of advances in nonlinear sensitivity analysis," in *System Modeling and Optimization: Proceedings of the 10th IFIP Conference New York City, USA, August 31–September 4, 1981*. Springer, 2005, pp. 762–770.
- [17] S. Hochreiter, "The vanishing gradient problem during learning recurrent neural nets and problem solutions," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 6, no. 02, pp. 107–116, 1998.
- [18] J. Han and C. Moraga, "The influence of the sigmoid function parameters on the speed of back-propagation learning," in *International workshop on artificial neural networks*. Springer, 1995, pp. 195–201.
- [19] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [20] K. Fukushima, "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position," *Biological cybernetics*, vol. 36, no. 4, pp. 193–202, 1980.
- [21] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [22] N. Kalchbrenner, E. Grefenstette, and P. Blunsom, "A convolutional neural network for modelling sentences," *arXiv preprint arXiv:1404.2188*, 2014.
- [23] A. Vinokourov, N. Cristianini, and J. Shawe-Taylor, "Inferring a semantic representation of text via cross-language correlation analysis," *Advances in neural information processing systems*, vol. 15, 2002.
- [24] A. Lavelli, F. Sebastiani, and R. Zanolini, "Distributional term representations: an experimental comparison," in *Proceedings of the thirteenth ACM international conference on Information and knowledge management*, 2004, pp. 615–624.
- [25] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *Advances in neural information processing systems*, vol. 26, 2013.
- [26] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
- [27] T. Mikolov, W.-t. Yih, and G. Zweig, "Linguistic regularities in continuous space word representations," in *Proceedings of the 2013 conference of the north american chapter of the association for computational linguistics: Human language technologies*, 2013, pp. 746–751.
- [28] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," *Advances in neural information processing systems*, vol. 27, 2014.
- [29] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.
- [30] N. Kalchbrenner and P. Blunsom, "Recurrent continuous translation models," in *Proceedings of the 2013 conference on empirical methods in natural language processing*, 2013, pp. 1700–1709.
- [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30,

2017.

- [32] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [33] OpenAI and A. Pilipiszyn. (2021) GPT-3 powers the next generation of apps. [Online]. Available: <https://openai.com/index/gpt-3-apps/>
- [34] OpenAI. (2023) GPT-4. [Online]. Available: <https://openai.com/index/gpt-4-research/>
- [35] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [36] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [37] S. Warren and L. Brandeis, "The right to privacy," in *Killing the Messenger: 100 Years of Media Criticism*. Columbia University Press, 1989, pp. 1–21.
- [38] A. Easton, "Records computers and the rights of citizens," *JMR, Journal of Marketing Research (pre-1986)*, vol. 12, no. 000003, p. 372, 1975.
- [39] N. Sadeh, A. Acquisti, T. D. Breaux, L. F. Cranor, A. M. McDonald, J. R. Reidenberg, N. A. Smith, F. Liu, N. C. Russell, F. Schaub *et al.*, "The usable privacy policy project," in *Technical report, Technical Report, CMU-ISR-13-119*. Carnegie Mellon University, 2013.
- [40] S. Wilson, F. Schaub, A. A. Dara, F. Liu, S. Cherivirala, P. G. Leon, M. S. Andersen, S. Zimmeck, K. M. Sathyendra, N. C. Russell *et al.*, "The creation and analysis of a website privacy policy corpus," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2016, pp. 1330–1340.
- [41] S. Zimmeck, Z. Wang, L. Zou, R. Iyengar, B. Liu, F. Schaub, S. Wilson, N. Sadeh, S. Bellovin, and J. Reidenberg, "Automated analysis of privacy requirements for mobile apps," in *2016 AAAI Fall Symposium Series*, 2016.
- [42] K. M. Sathyendra, S. Wilson, F. Schaub, S. Zimmeck, and N. Sadeh, "Identifying the provision of choices in privacy policy text," in *Proceedings of the 2017 conference on empirical methods in natural language processing*, 2017, pp. 2774–2779.
- [43] D. A. Audich, R. Dara, and B. Nonnecke, "Privacy policy annotation for semi-automated analysis: A cost-effective approach," in *Trust Management XII: 12th IFIP WG 11.11 International Conference, IFIPTM 2018, Toronto, ON, Canada, July 10–13, 2018, Proceedings 12*. Springer, 2018, pp. 29–44.
- [44] F. Liu, S. Wilson, P. Story, S. Zimmeck, and N. Sadeh, "Towards automatic classification of privacy policy text," *School of Computer Science Carnegie Mellon University*, 2018.
- [45] H. Harkous, K. Fawaz, R. Lebrete, F. Schaub, K. G. Shin, and K. Aberer, "Polisis: Automated analysis and presentation of privacy policies using deep learning," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 531–548.
- [46] R. N. Zaeem, R. L. German, and K. S. Barber, "Privacycheck: Automatic summarization of privacy policies using data mining," *ACM Transactions on Internet Technology (TOIT)*, vol. 18, no. 4, pp. 1–18, 2018.
- [47] P. Story, S. Zimmeck, A. Ravichander, D. Smullen, Z. Wang, J. Reidenberg, N. C. Russell, and

- N. Sadeh, "Natural language processing for mobile app privacy compliance," in *AAAI spring symposium on privacy-enhancing artificial intelligence and language technologies*, vol. 2, no. 4, 2019, p. 4.
- [48] S. Zimmeck, P. Story, D. Smullen, A. Ravichander, Z. Wang, J. Reidenberg, N. C. Russell, and N. Sadeh, "Maps: Scaling privacy compliance analysis to a million apps," *Proceedings on Privacy Enhancing Technologies*, 2019.
- [49] M. Fan, L. Yu, S. Chen, H. Zhou, X. Luo, S. Li, Y. Liu, J. Liu, and T. Liu, "An empirical evaluation of gdpr compliance violations in android mhealth apps," in *2020 IEEE 31st international symposium on software reliability engineering (ISSRE)*. IEEE, 2020, pp. 253–264.
- [50] L. Yu, X. Luo, J. Chen, H. Zhou, T. Zhang, H. Chang, and H. K. Leung, "Ppchecker: Towards accessing the trustworthiness of android apps' privacy policies," *IEEE Transactions on Software Engineering*, vol. 47, no. 2, pp. 221–242, 2018.
- [51] D. Bui, Y. Yao, K. G. Shin, J.-M. Choi, and J. Shin, "Consistency analysis of data-usage purposes in mobile apps," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2824–2843.
- [52] A. Alabduljabbar, A. Abusnaina, Ü. Meteriz-Yildiran, and D. Mohaisen, "Automated privacy policy annotation with information highlighting made practical using deep representations," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2378–2380.
- [53] A. Alabduljabbar and D. Mohaisen, "Measuring the privacy dimension of free content websites through automated privacy policy analysis and annotation," in *Companion Proceedings of the Web Conference 2022*, 2022, pp. 860–867.
- [54] E. Grünewald and F. Pallas, "Tilt: A gdpr-aligned transparency information language and toolkit for practical privacy engineering," in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, 2021, pp. 636–646.
- [55] M. Gebauer, F. Maschhur, N. Leschke, E. Grünewald, and F. Pallas, "A 'human-in-the-loop' approach for information extraction from privacy policies under data scarcity," in *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2023, pp. 76–83.
- [56] C. Tang, Z. Liu, C. Ma, Z. Wu, Y. Li, W. Liu, D. Zhu, Q. Li, X. Li, T. Liu *et al.*, "Policygpt: Automated analysis of privacy policies with large language models," *arXiv preprint arXiv:2309.10238*, 2023.
- [57] L. Garza, L. Elluri, A. Kotal, A. Piplai, D. Gupta, and A. Joshi, "PrivComp-KG: Leveraging Knowledge Graph and Large Language Models for Privacy Policy Compliance Verification," *arXiv preprint arXiv:2404.19744*, 2024.
- [58] S. Arora, H. Hosseini, C. Utz, V. K. Bannihatti, T. Dhellemmes, A. Ravichander, P. Story, J. Mangat, R. Chen, M. Degeling *et al.*, "A tale of two regulatory regimes: Creation and analysis of a bilingual privacy policy corpus," in *LREC proceedings*, 2022.
- [59] WSTS and SIA. (2024) Semiconductor market revenue worldwide from 1987 to 2025 (in billion u.s. dollars). [Online]. Available: <https://www.statista.com/statistics/266973/global-semiconductor-sales-since-1988/>
- [60] Nvidia. (2024) Nvidia revenue worldwide from fiscal year 2015 to 2024 (in billion u.s. dollars).

- [Online]. Available: <https://www.statista.com/statistics/988030/nvidia-revenue-worldwide/>
- [61] Ollama. (2024) Ollama: GitHub repository. [Online]. Available: <https://github.com/ollama/ollama>
- [62] H. Face. (2024) :huggingface: Transformers. [Online]. Available: <https://huggingface.co/docs/transformers/index>
- [63] Radim Řehůřek. (2023) Gensim Github Repository. [Online]. Available: <https://github.com/piskvorky/gensim>
- [64] Facebook AI Research. (2024) fastText Github Repository. [Online]. Available: <https://github.com/facebookresearch/fastText>
- [65] RapidFuzz. (2024) RapidFuzz Github Repository. [Online]. Available: <https://github.com/rapidfuzz/RapidFuzz>
- [66] SeatGeek. (2024) FuzzyWuzzy Github Repository. [Online]. Available: <https://github.com/seatgeek/fuzzywuzzy/tree/4bf28161f7005f3aa9d4d931455ac55126918df7>
- [67] V. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Proceedings of the Soviet physics doklady*, 1966.
- [68] FastText. (2024) FastText Website. [Online]. Available: <https://fasttext.cc/docs/en/english-vectors.html>

Appendices

Appendix 1: Listings

1 Unannotated Passage

Listing 1: Sample Unannotated Passage as a JSON Object

```
{
  "type": "list_item",
  "context": [
    {
      "text": "ABC.com Privacy Notice",
      "type": "h1"
    },
    {
      "text": "What Personal Information About Customers Does ABC Collect?",
      "type": "h2"
    },
    {
      "text": "Here are the types of personal information we collect:",
      "type": "list_intro"
    }
  ],
  "passage": "Information from Other Sources: We might receive information about you from other sources, such as updated delivery and address information from our carriers, which we use to correct our records and deliver your next purchase more easily. We do not, however, collect your gender and sexual orientation. Click here to see additional examples of the information we receive. This information is stored on servers in the USA. You may request that we delete your personal information at any time."
}
```

2 Annotated Passage

Listing 2: Sample Passage with Annotations as a JSON Object

```
{
  "type": "list_item",
  "context": [
    {
      "text": "ABC.com Privacy Notice",
      "type": "h1"
    },
    {
      "text": "What Personal Information About Customers Does ABC Collect?",
      "type": "h2"
    },
    {
      "text": "Here are the types of personal information we collect:",
      "type": "list_intro"
    }
  ],
  "passage": "Information from Other Sources: We might receive information about you from other sources, such as updated delivery and address information from our carriers, which we use to correct our records and deliver your next purchase more easily. We do not, however, collect your gender and sexual orientation. Click here to see additional examples of the information we receive. This information is stored on servers in the USA. You may request that we delete your personal information at any time.",
  "annotations": [
    {
      "requirement": "Data Categories",
      "value": "delivery and address information",
      "generalized_value": "contact information",
      "performed": true
    },
    {
      "requirement": "Source of Data",
      "value": "our carriers",
      "generalized_value": "third-party sources",
      "performed": true
    },
    {
      "requirement": "Processing Purpose",
      "value": "correct our records",
      "generalized_value": "data accuracy",
      "performed": true
    }
  ]
}
```

```
    },  
    {  
      "requirement": "Processing Purpose",  
      "value": "deliver your next purchase more easily",  
      "generalized_value": "service improvement",  
      "performed": true  
    }  
  ]  
}
```


3 Reviewed Passage

Listing 3: Sample Passage with Reviewed Annotations as a JSON Object

```
{
  "review": "The annotations are mostly accurate but have some minor
    omissions and need refinement. The identified data categories ('
    delivery and address information') and their source ('our carriers')
    are correctly annotated. The processing purposes are accurately
    captured as well. However, the annotation misses the data storage
    location ('stored on servers in the USA') which pertains to the
    requirement 'Third-country Transfers'. Additionally, the annotation
    should include the user's right to request data deletion, which
    aligns with the 'Right to Erasure' requirement. The mention that
    gender and sexual orientation are not collected is correctly omitted
    from the annotation, as it is not relevant to the annotated
    categories.",
  "score": 0.8,
  "revised": {
    "type": "list_item",
    "context": [
      {
        "text": "ABC.com Privacy Notice",
        "type": "h1"
      },
      {
        "text": "What Personal Information About Customers Does ABC
          Collect?",
        "type": "h2"
      },
      {
        "text": "Here are the types of personal information we collect:",
        "type": "list_intro"
      }
    ]
  },
  "passage": "Information from Other Sources: We might receive
    information about you from other sources, such as updated delivery
    and address information from our carriers, which we use to
    correct our records and deliver your next purchase more easily. We
    do not, however, collect your gender and sexual orientation.
    Click here to see additional examples of the information we
    receive. This information is stored on servers in the USA. You may
    request that we delete your personal information at any time.",
  "annotations": [
    {
      "requirement": "Data Categories",
      "value": "delivery and address information",
```

```
    "generalized_value": "contact information",
    "performed": true
  },
  {
    "requirement": "Source of Data",
    "value": "our carriers",
    "generalized_value": "third-party sources",
    "performed": true
  },
  {
    "requirement": "Processing Purpose",
    "value": "correct our records",
    "generalized_value": "data accuracy",
    "performed": true
  },
  {
    "requirement": "Processing Purpose",
    "value": "deliver your next purchase more easily",
    "generalized_value": "service improvement",
    "performed": true
  },
  {
    "requirement": "Third-country Transfers",
    "value": "stored on servers in the USA",
    "generalized_value": "data storage location",
    "performed": true
  },
  {
    "requirement": "Right to Erasure",
    "value": "request that we delete your personal information at any
      time",
    "generalized_value": "data deletion request",
    "performed": true
  }
]
}
```

Appendix 2: Tables

1 Complete Metrics

For Table 1, the Average Correct Requirements Ratio is abbreviated as "ACRR", Distinct Precision is DP, Distinct Recall is DR and Distinct F1-Score is DF1.

Table 1: Metrics from Annotated Dataset

Model	ACRR	Precision	Recall	F1 Score	DP	DR	DF1
llama3.1_8b-instruct-q4_0	0.5721	0.3268	0.5840	0.4191	0.2667	0.7708	0.3962
llama3.1_8b-instruct-fp16	0.5902	0.3449	0.6125	0.4413	0.2784	0.7878	0.4114
gemma2_9b-instruct-q4_0	0.6400	0.5984	0.6760	0.6348	0.5054	0.7251	0.5956
gemma2_27b-instruct-q4_0	0.6212	0.5561	0.6531	0.6007	0.4792	0.7925	0.5972
mistral-nemo_12b-instruct-2407-q4_0	0.6013	0.4998	0.5630	0.5296	0.4402	0.7884	0.5650
mixtral_instruct	0.5728	0.4096	0.5043	0.4521	0.3603	0.7871	0.4947
phi3_14b-medium-128k-instruct-q4_0	0.5269	0.3478	0.3911	0.3681	0.3264	0.5905	0.4255
qwen2_7b-instruct-fp16	0.5205	0.2871	0.5062	0.3664	0.2094	0.6408	0.3127

For Table 2, the same abbreviations are used as in Chapter 5, Section 5.2.1.3.

Table 2: Similarity Scores from Annotated Dataset

Model	ASS	AWLS	ASSM	AWLSSM
llama3.1_8b-instruct-q4_0	0.9931	0.8103	0.2633	0.2148
llama3.1_8b-instruct-fp16	0.9926	0.8073	0.2810	0.2285
gemma2_9b-instruct-q4_0	0.9957	0.8724	0.4630	0.4057
gemma2_27b-instruct-q4_0	0.9953	0.8486	0.4273	0.3643
mistral-nemo_12b-instruct-2407-q4_0	0.9936	0.8078	0.3578	0.2909
mixtral_instruct	0.9912	0.7741	0.2899	0.2261
phi3_14b-medium-128k-instruct-q4_0	0.9891	0.7025	0.2231	0.1585
qwen2_7b-instruct-fp16	0.9913	0.7581	0.2223	0.1700

For the abbreviations used in Table 3, see the description of Table 1.

Table 3: Metrics from Reviewed Dataset

Model	ACRR	Precision	Recall	F1 Score	DP	DR	DF1
llama3.1_8b-instruct-q4_0	0.6147	0.4218	0.5187	0.4652	0.3579	0.6487	0.4613
llama3.1_8b-instruct-fp16	0.6389	0.4879	0.4654	0.4764	0.4032	0.5301	0.4580
gemma2_9b-instruct-q4_0	0.6350	0.5774	0.5706	0.5740	0.4765	0.6074	0.5340
gemma2_27b-instruct-q4_0	0.6415	0.5333	0.5158	0.5244	0.4604	0.6786	0.5486
mistral-nemo_12b-instruct-2407-q4_0	0.5937	0.4401	0.4891	0.4633	0.3937	0.6272	0.4837
mixtral_instruct	0.5566	0.3871	0.4385	0.4112	0.3426	0.6107	0.4390
phi3_14b-medium-128k-instruct-q4_0	0.5269	0.3492	0.3886	0.3678	0.3293	0.6013	0.4255
qwen2_7b-instruct-fp16	0.5205	0.1785	0.4101	0.2487	0.1673	0.6712	0.2678

For Table 4, the abbreviations used are the same as for Table 2.

Table 4: Similarity Scores from Reviewed Dataset

Model	ASS	AWLS	ASSM	AWLSSM
llama3.1_8b-instruct-q4_0	0.9913	0.7877	0.3005	0.2388
llama3.1_8b-instruct-fp16	0.9907	0.7888	0.3098	0.2466
gemma2_9b-instruct-q4_0	0.9934	0.8276	0.3998	0.3331
gemma2_27b-instruct-q4_0	0.9933	0.8023	0.3530	0.2851
mistral-nemo_12b-instruct-2407-q4_0	0.9918	0.8067	0.2990	0.2432
mixtral_instruct	0.9912	0.7811	0.2565	0.2021
phi3_14b-medium-128k-instruct-q4_0	0.9897	0.7369	0.2230	0.1661
qwen2_7b-instruct-fp16	0.9891	0.7285	0.1405	0.1035

Appendix 3: Figures

1 Performance Metrics (Reviewed Annotations)

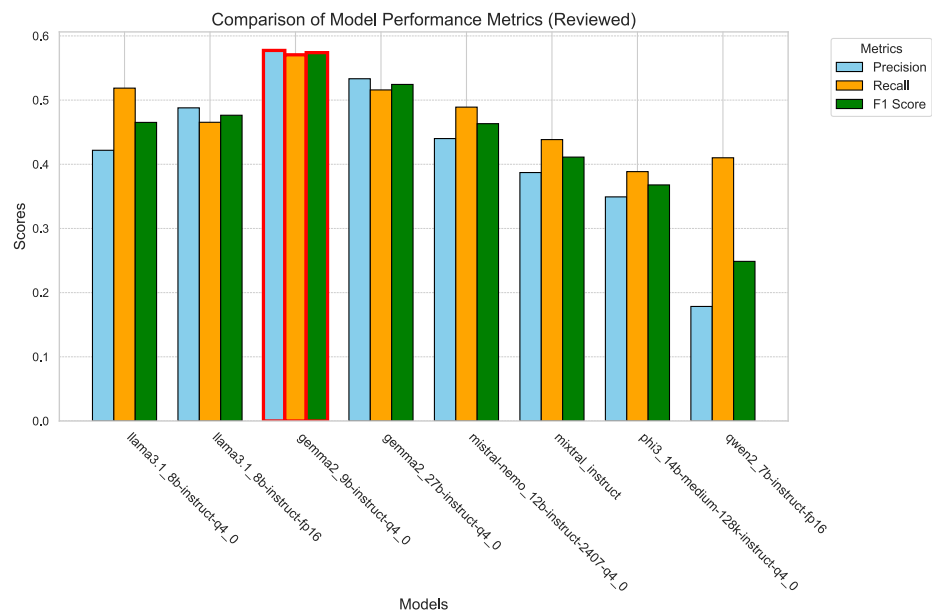


Figure 1: Performance Metrics (Reviewed Annotations)

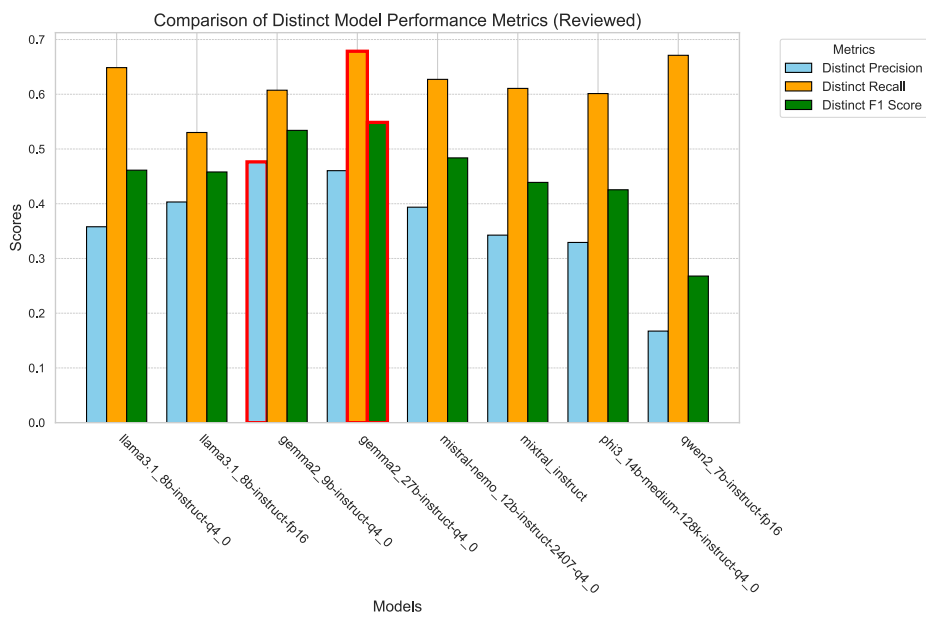


Figure 2: Distinct Performance Metrics (Reviewed Annotations)

2 F1-Score Sorted by Model Size

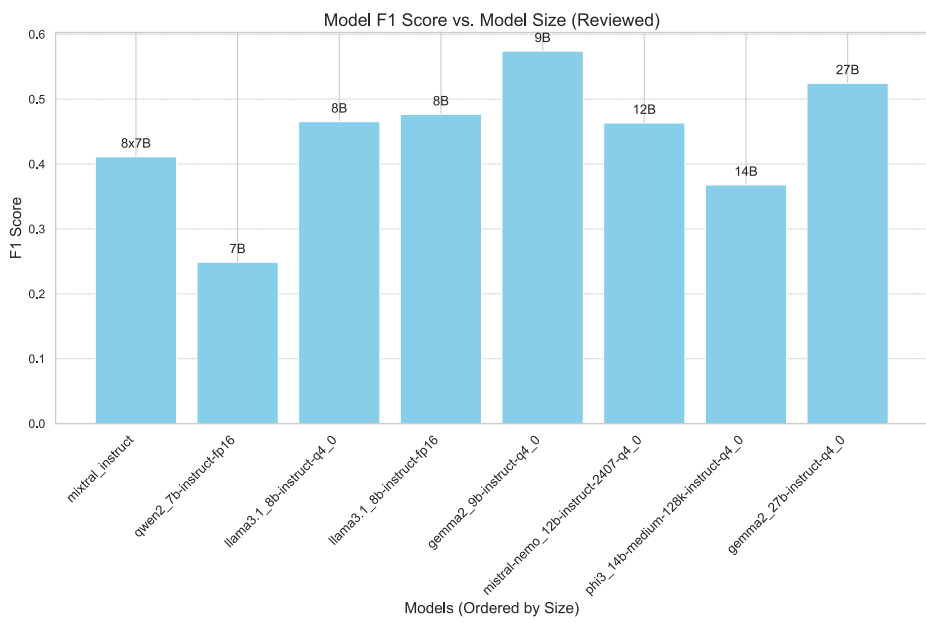


Figure 3: Model F1-Scores, Sorted by Model Size (Reviewed Annotations)

3 Average Similarity Scores

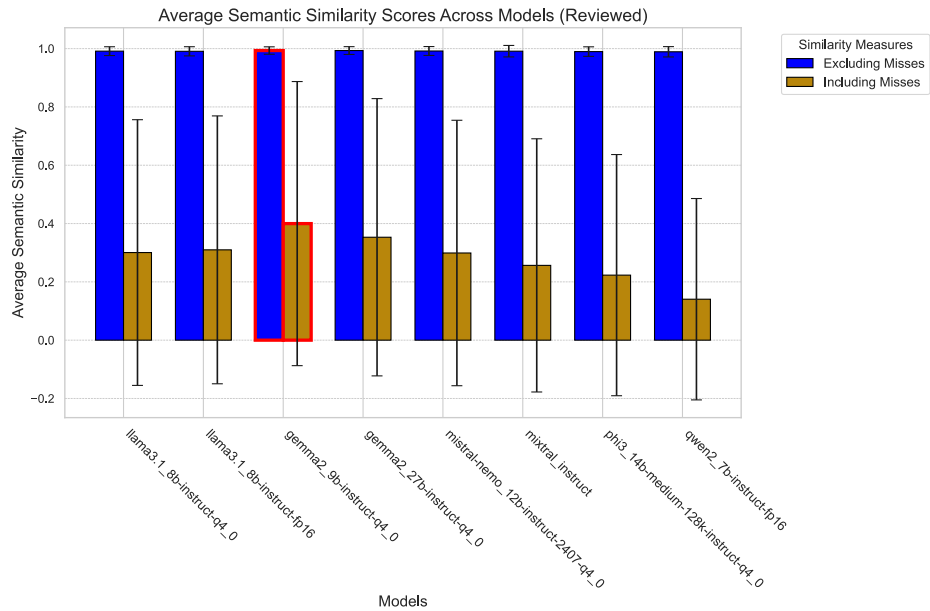


Figure 4: Average Semantic Similarity Scores (with Standard Deviation) (Reviewed Annotations)

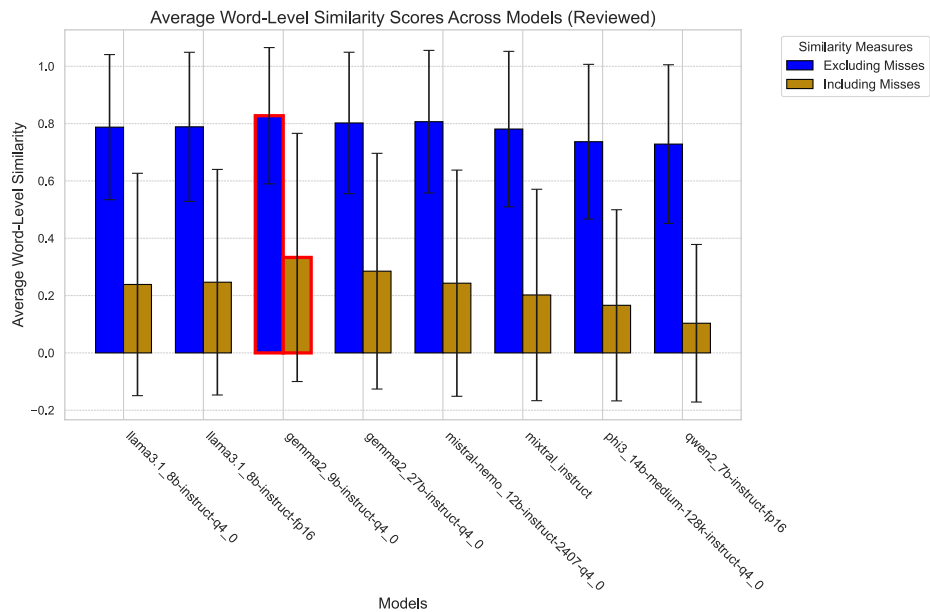


Figure 5: Average Word-Level Similarity Scores (with Standard Deviation) (Reviewed Annotations)

4 Correct Requirements Ratio (Reviewed Annotations)

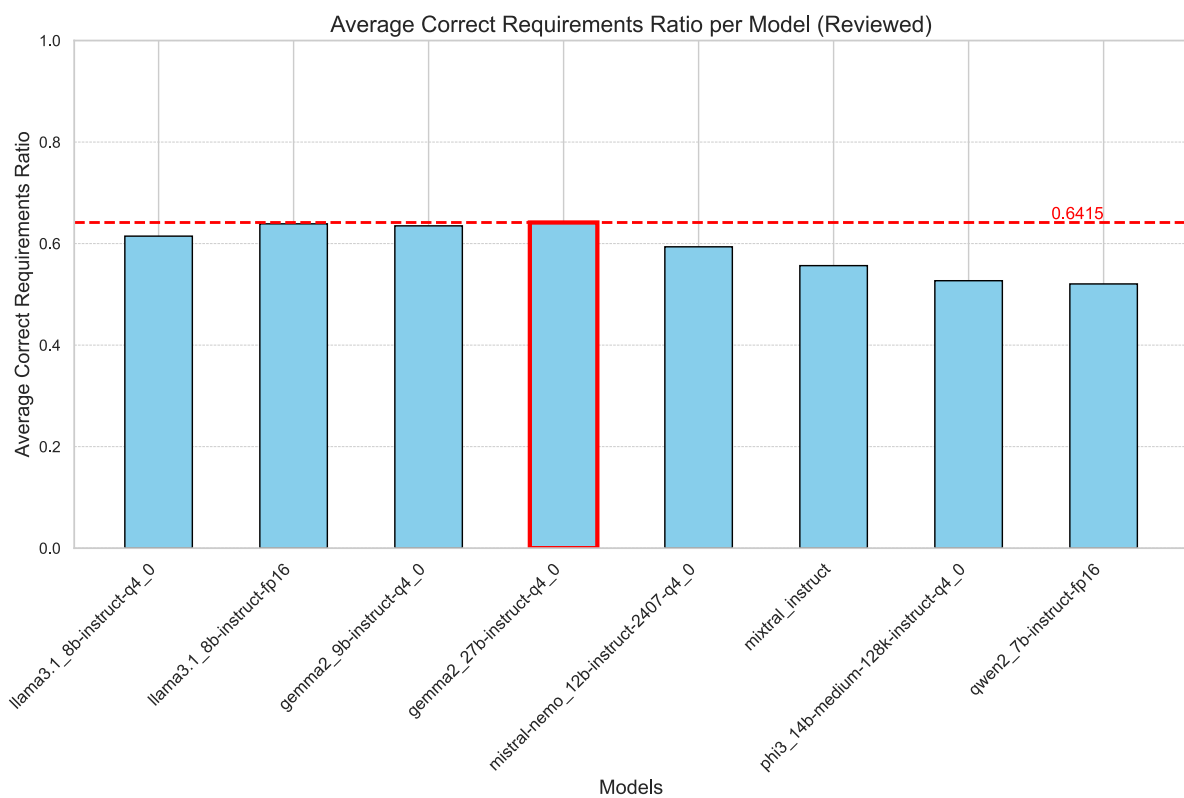


Figure 6: Average Correct Requirements Ratio per Model (Reviewed Annotations)

5 Confusion Matrices (Reviewed Annotations)

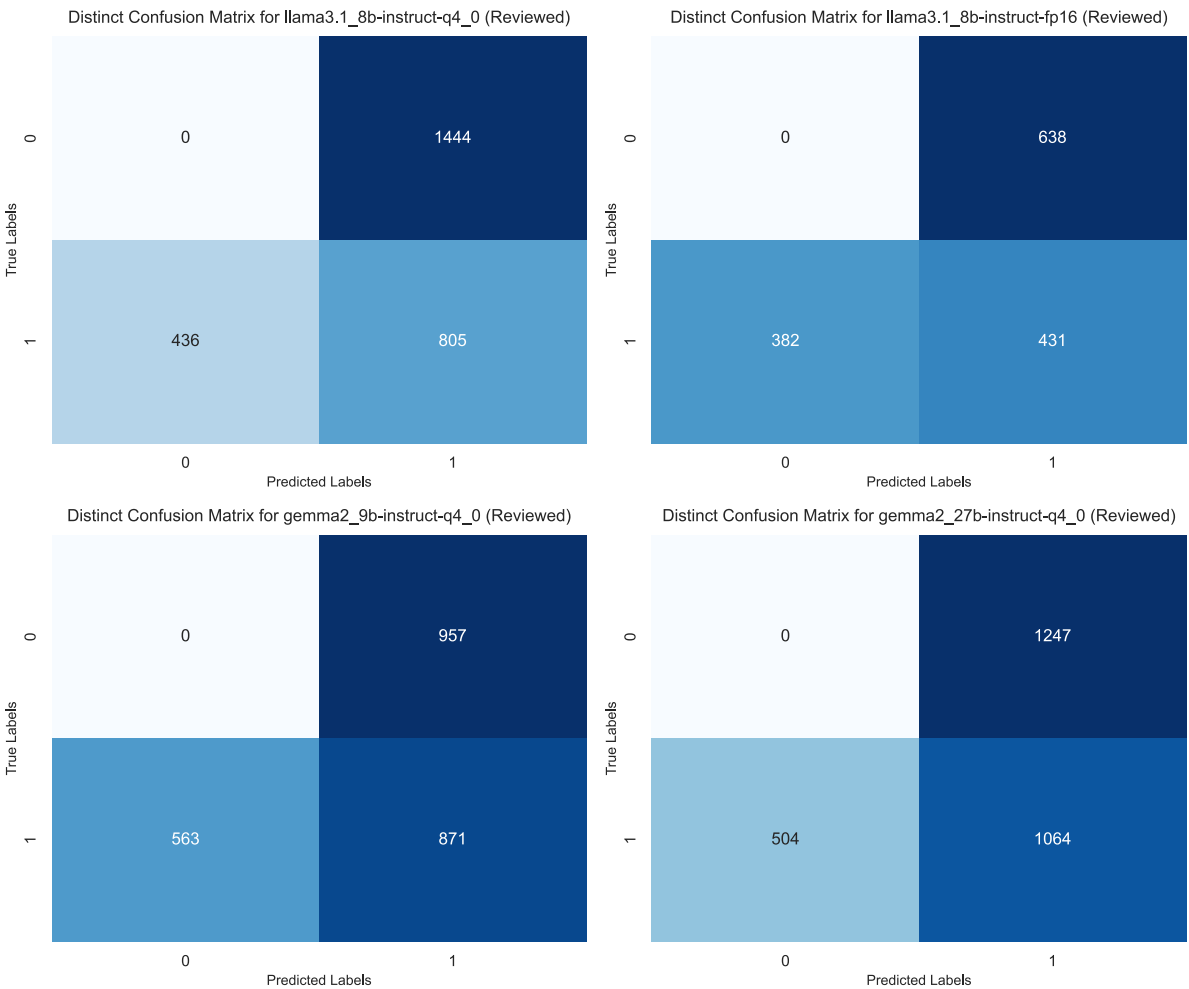


Figure 7: Distinct Confusion Matrices: Part 1 (Reviewed Annotations)

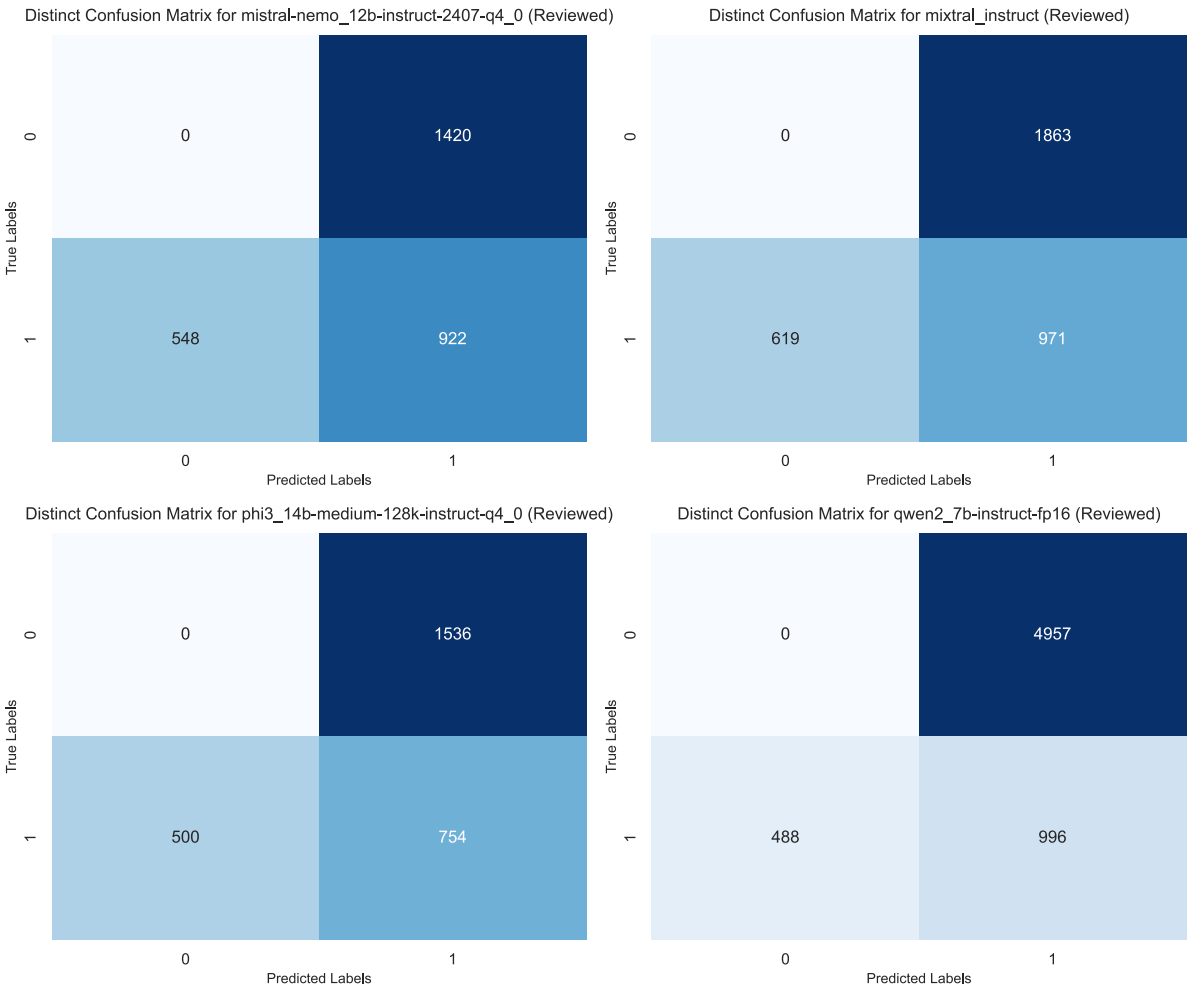


Figure 8: Distinct Confusion Matrices: Part 2 (Reviewed Annotations)

6 Combined Metrics

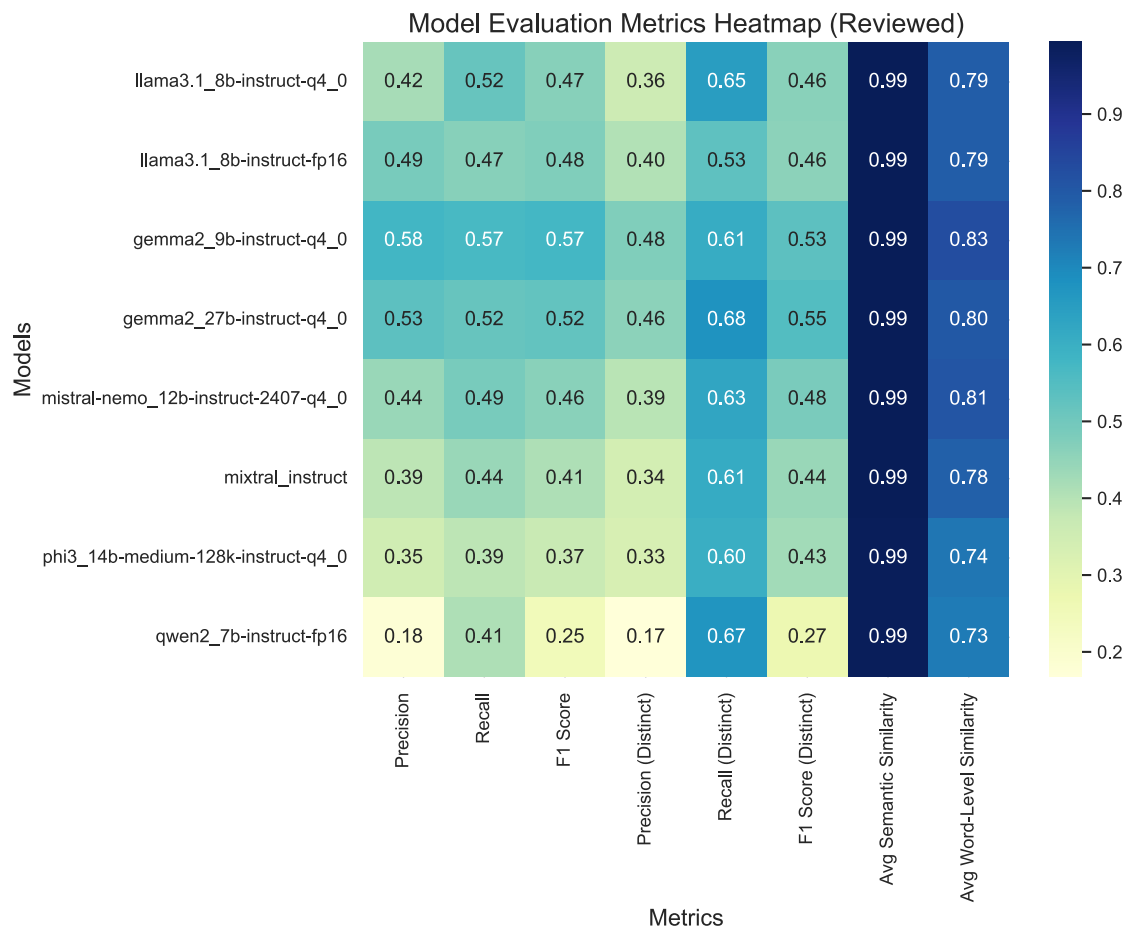


Figure 9: Model Metric Heat Map (Reviewed Annotations)